FRP 2026 AI ENGINEERING TRACK DAY 6

How AI models learn

Reading, school and practice

Vladimir Albrekht

ISSAI, Nazarbayev University, 29 October 2026

START

Day 6 starts with a question. Think about it alone for 10 seconds, then say your idea.

ChatGPT writes a poem in Kazakh.

Who taught it?

> **Nobody wrote its rules by hand. Today you see how an AI model learns.**

LIVE ON THE ISSAI SERVER (SCREENSHOT: A REAL RUN, 16 OCT)

Live: a model is born

We start a new, tiny model on the class GPU server. It reads Kazakh Wikipedia while we talk.

1 Send it to the GPU: sbatch puts the job in the queue, tail -F shows its log

```
student@frp-stu-s92:~/work/day6$ sbatch --wrap "python3 -u tiny_gpt.py --data data/clean.txt --out runs/live2"
Submitted job 29 (limit 15 min, 0 job(s) before you). Output: ~/work/jobs/29.log
student@frp-stu-s92:~/work/day6$ tail -F ~/work/jobs/29.log
tail: cannot open '/home/student/work/jobs/29.log' for reading: No such file or directory
tail: '/home/student/work/jobs/29.log' has appeared;  following new file
[job 29 on GPU 9, limit 15 min, started 11:48:08]
params 10.87 M, alphabet 165, train chars 60.0 M, device cuda

step      0   loss 5.29       0 s
   ҚазақстаніЕЫцuAE     ұАв[2vыАюХф,“o -тҚәВи)R²PrkП?үЛыЧнҚжаыа7T0L0зIVұюD«Ю0tйу,қЮ?хө¥mfzЯГPisфyv8нД²y²3Л;iS.9iCXL—О«дд
нуЦ/zО7БЯпфГгСМ:uң5ҚлГ<T+3V'VwБХ<0wzоз  hCgыаУ¥Лю
```

2 Step 0 (no learning yet): random letters. Its 11 million numbers are random.

> **A new model knows nothing. We check it again in 20 minutes.**

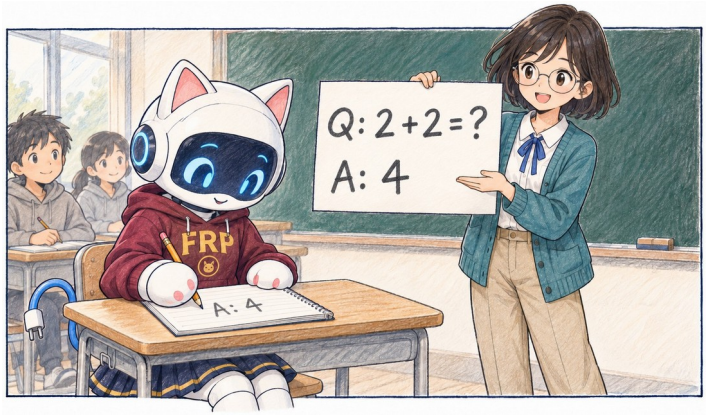
A model grows up in 3 stages

Every big model, like ChatGPT, learns in 3 stages: Parts A, B, C. Then: **D** how this class trained, **E** how trainings differ, **F** your turn. The robot cat = the AI.



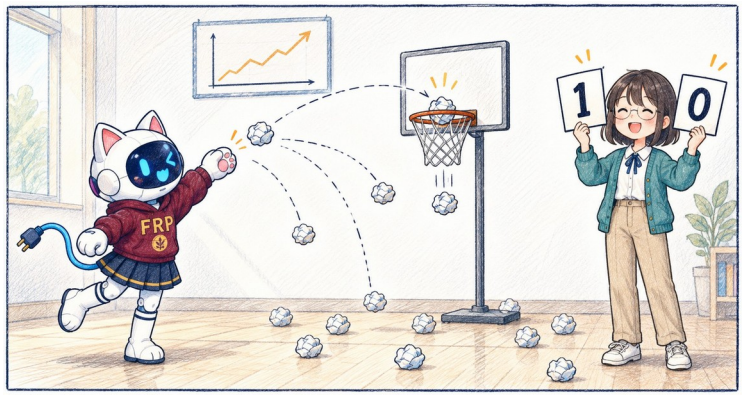
A. Reading

pretraining



B. School

fine-tuning



C. Practice

reinforcement learning

Today we do all 3 stages on our server. Then you train one.

6 words for today

These 6 words come again and again today. If you forget one, look back at this slide.

knob

a number inside the model; learning turns it

token

a piece of a word; models read pieces

loss

the mistake score (lower = better)

GPU

a chip for millions of sums at once

fine-tuning

school: more training on example answers

RL

reinforcement learning: practice with a score

Learning = the model turns its knobs to make its loss lower.

Trainings in this class this week

This week, 7 projects of this class used AI. Orange cards: the project trained a model. White cards: no training. V100 = the GPU in our class server.

scam-detector

fine-tuning + a new head
a ready model learns safe vs scam: 3.4 min on 1 V100

Tetris

reinforcement learning
plays 1,064 games, reward = lines: 23 min on 1 V100

song-recommender

no training
a ready model turns each song into 512 numbers

solar-forecast

tiny models, 30 days
learn from June data, tested on July

school-video

no training
prompts + an AI that checks the videos

quiz-bank, university-guide

no training
quizzes from the class model; a web agent

Different projects need different kinds of training, or none.

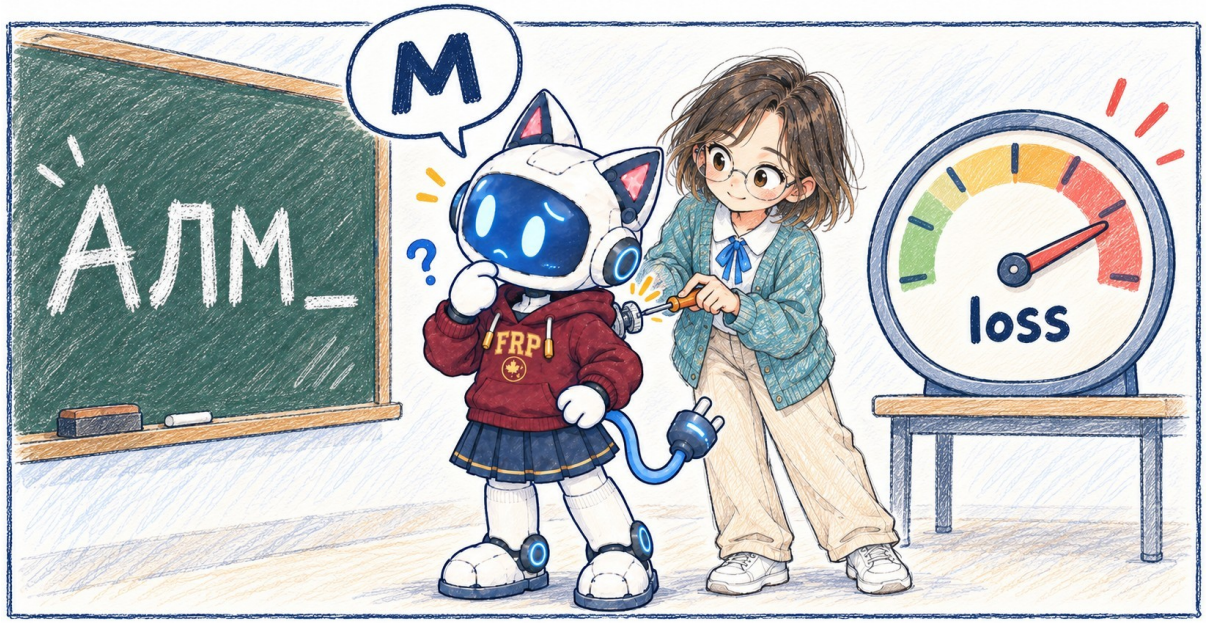
PART A: READING EVERYTHING

Stage 1: it reads everything

Pretraining: guess the next letter, billions of times.

Learning = guess, check, turn the knobs

Stage 1 is reading. The model plays one game again and again: guess the next letter.



1. Guess: the AI (robot cat) sees “Алм_” and guesses the next letter.

2. Check: Aru shows the real letter. The meter shows the loss: how wrong it was.

3. Turn: every knob turns a tiny bit. The right letter “a” had a 10 % chance; after 1 step: 11 %.

After a wrong guess, every knob turns a tiny bit toward the right letter.

PART A: READING

ACTIVITY, 4 MINUTES

You are the model: guess the next letter

Alone, on paper: write the missing letter of each word. Then hands up if you were wrong. We count the wrong guesses.

1. **Алм_**
2. **Қазақста_**
3. **Абай Құнанбай_лы**

> **The number of wrong guesses is the loss. A big loss means a big surprise.**

Loss goes down = it learns

Our tiny model (10.9 million knobs) read Kazakh Wikipedia for 7 minutes on 1 GPU (V100), 14 Oct. The line = its loss after each training step.



step = 1 round of guess, check, turn the knobs

loss = the average surprise per letter, lower = better

loss 5 = guessing blind

loss 1 = almost sure

The loss fell from 5.29 to 0.88 in 12,000 steps: the model learned.

REAL TEXT AFTER “ҚАЗАҚСТАН”

Watch it learn (1)

Our tiny model continued the word “Қазақстан” at 3 moments in its training. Real texts.

step 0 ҚазақстаніЕЫіцuАЕ ұАв[2vыАюХф,“о -тҚәВи)R²PrkП? үЛыЧнҚжаыа7T0LӨзIVұюD«Ютйұ,қЮ …

step 25 Қазақстане жанысоманертыліги, ок тілыңа а қа жарың жасылг сысентыматынар, …

step 400 Қазақстан ашқын **жұмыс** тылдау, оны **бәрі** метін жүрден, арақ сауындағы, бар, …

> **First it learned the letters, then words. Nobody told it what a word is.**

Watch it learn (2)

The same model later: step 800, step 1,500 and step 12,000 (the end, after 7 minutes).

step 800	Қазақстан Республикасы құрам, оқ-телезі атқару үшін жасай тау. Құрмет тұрқымдасының …
step 1,500	Қазақстан Республикасы құрамасының әкімшілік округіне бағынады. 2001 жылдың 5 …
step 12,000	Қазақстан ЕО-ның 110 мың халық кешені, Дағыстандық Қазақстанның ұлттық құқықтары мен …

One text can be lucky or unlucky. The loss is the average of millions of guesses.

> **Good Kazakh, invented facts: it learned how Kazakh sounds, not what is true.**

LIVE: THE TERMINAL (SCREENSHOT: A REAL RUN, 16 OCT)

Our model from the start: now

The model we started on slide 3 has finished: 12,000 steps. Here is the end of its log.

1 The end of the log: tail -n 14 ~/work/jobs/28.log

```
student@frp-stu-s92:~/work/day6$ tail -n 14 ~/work/jobs/28.log

 step   1500   loss 1.14      62 s
    Қазақстан Республикасы қоректі құралдар атқару үшін, алай тау-кен құрай, қоректеніп, ішкі қа
 тарға сол құраладарды аң бой отарлай болды. Тау-кен беріле адам әсер

 step   3000   loss 1.03     118 s
    Қазақстан Республикасы құрамына кіретін атқару үшін, алайда сентриториялдық құралдардің ең і
 рі жер ретінде сенімді. Ерей олардың бір түрі қарапайым кезінде «Қаз

 step   6000   loss 0.95     230 s
    Қазақстан Республикасы Парламенті Сенатының депутаттары / Сосновные мемлекеттік қайраткерлер
 і /  / Куғия — Ресейдегі өзен. Оренбург облысы, Тамбов облысы, Тамбов облы

 step  12000   loss 0.88     452 s
    Қазақстан ЛКЖО Егіншілік хатшысы тіл білімінің дипломанты қызметін атқарды. / «Құрмет белгіс
 і» орденімен марапатталған. / Дереккөздер / Қазақстан мұнайшылары / Тұлғалар
 done 452.3 s
 [job 28 finished, exit code 0]
student@frp-stu-s92:~/work/day6$
```

step 0: loss 5.29

random letters

step 12,000: loss 0.88

Kazakh words, after 7.5 minutes

A real run in a class workspace, 16 Oct.

In 7.5 minutes: from random letters (loss 5.29) to Kazakh words (loss 0.88).

Where does the text come from?

To learn a language, a model must read a lot of text. How much? Our tiny model next to a big one.



60 million

letters: what our tiny model read

300 billion

pages in Common Crawl, a copy of the web

36 trillion

tokens read by Qwen3, a big open model

> **Reading one book a day, you need 14,000 lifetimes to read what Qwen3 read.**

ACTIVITY: HANDS UP, A OR B

Which model read dirty data?

Model 1 read clean Wikipedia. Model 2 read Wikipedia + web junk (menus, ads). Read both texts alone, 1 minute. Hands up: which text is from model 2?

Text A

/ Көне Асыл Асыл қаласы () – Үндістанды асуының бір бөлігі, екі жасқа және одан да көп мүсіндер әшекейленеді. Асыл Асыл ...

Text B

/ Біз cookie файлдарын қолданамыз. Келісемін / Біз cookie файлдарын қолданамыз. Келісемін / 2024 Барлық құқықтар ...

Both models had the same size and the same steps. Only the data was different.

Junk in, junk out

Each model wrote 50 texts. We counted the texts with junk in them. We also measured each loss on clean text.



texts with junk, of 50 texts from each model (lower = better)

Loss on clean text

(lower = better)

clean 0.88

dirty 0.94

The model copies what it reads, also the junk. And junk wastes its practice time.

Clean the data first

Big teams spend a lot of time cleaning data. Picture: Aru filters web pages; junk goes to the bin.



FineWeb-Edu, 2024 (Hugging Face)

kept only the educational web pages

A model trained on it got the same exam score (MMLU) with

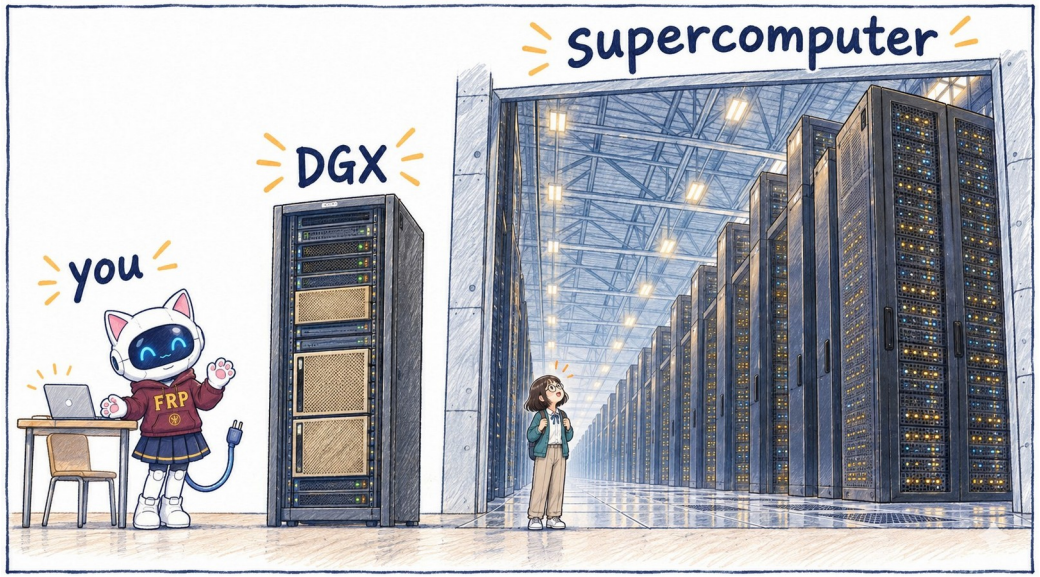
38 billion

tokens. Another dataset needed 300 billion.

Look at your data before you train on it.

Why training needs a supercomputer

How much computer work is training? The big model here: 27 billion knobs, 36 trillion tokens.



work ≈ 6 × knobs × tokens

7 minutes

our tiny model (11 million knobs), 1 GPU

≈ 230 years

big model, our DGX server (16 GPUs)

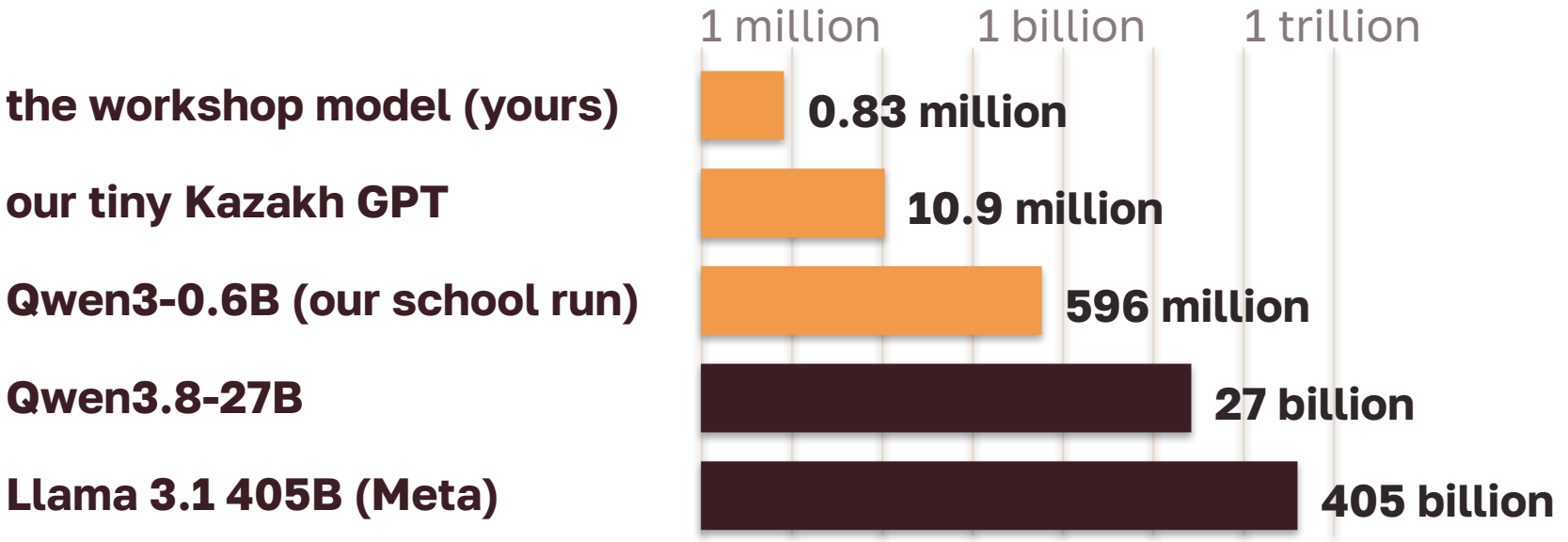
≈ 3 weeks

big model, 8,000 of the newest GPUs

> **That is why we usually start from a model that others trained.**

How many knobs does a model have?

Training changes only the knobs: the numbers inside the model. The program around them stays the same.



NEW WORD

weight = one knob

NEW WORD

parameter = a knob, too

Orange = models we trained. Every grey line = 10 times more knobs.

Learning = changing knobs. Models have millions to billions.

Learning rate: how far each knob turns

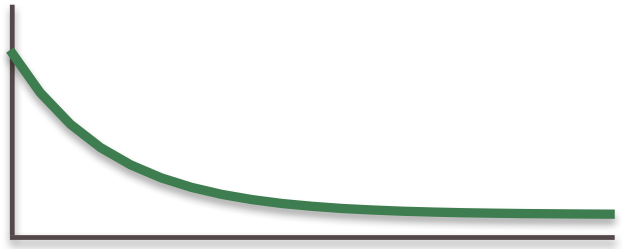
After each step every knob turns a little. How far? You choose it before training. Each panel: loss (lower is better) over training steps. Drawn to show the idea.

too small



slow: still high at the end

good



fast down, then settles

too big



jumps, can even go up

NEW WORD

learning rate = how far a knob turns in one step

In this class: the scam-detector 0.00002, the Tetris project 0.001

Too small is slow, too big jumps. Try a few values.

Batch, step, epoch

The scam-detector project trained on 66,160 messages. The model does not read them one by one, and not all at once, but in small groups.



NEW WORD

batch = 32 messages read together

NEW WORD

step = one knob turn after a batch

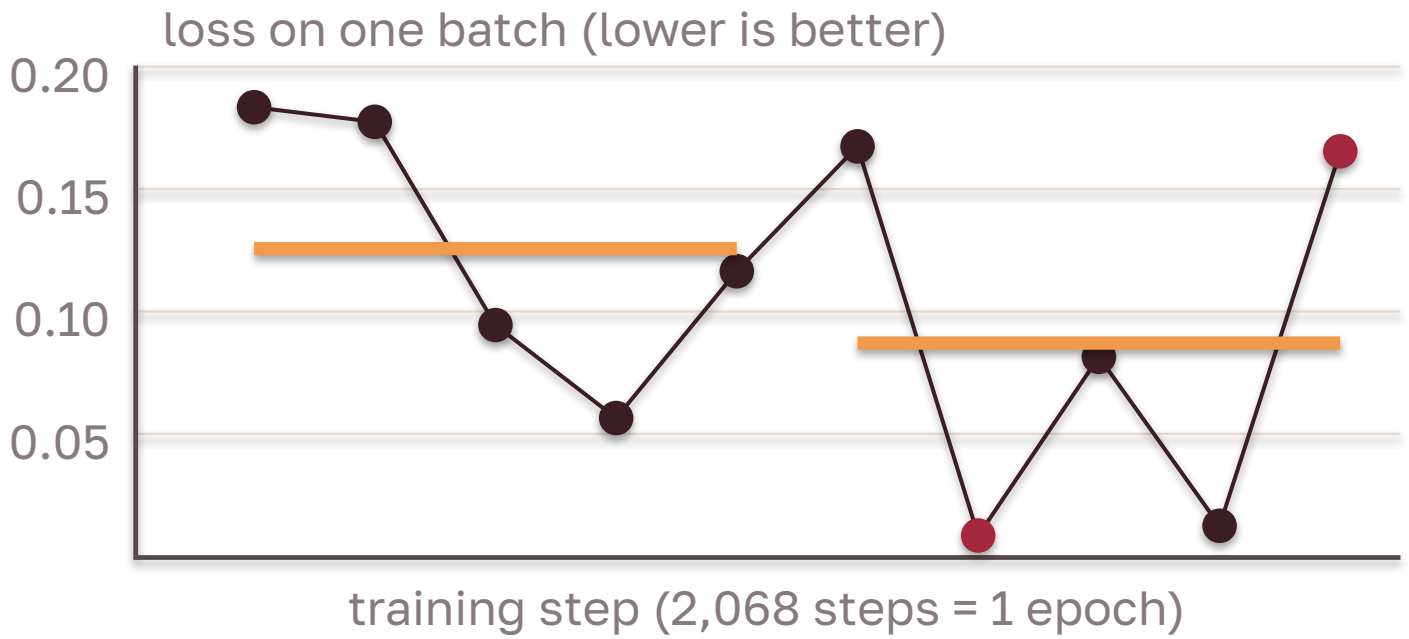
NEW WORD

epoch = all 66,160 messages once

66,160 ÷ 32 = 2,068 steps = 1 epoch = 3.4 minutes.

A real log: one batch is noisy

The scam-detector printed its loss every 200 steps. Each dot = the loss on ONE batch of 32 messages.



step 1,400: **0.008**

step 2,000: **0.165**

20 times worse? No: only a different batch.

Orange lines: average of 5 batches, **0.125 → 0.087**

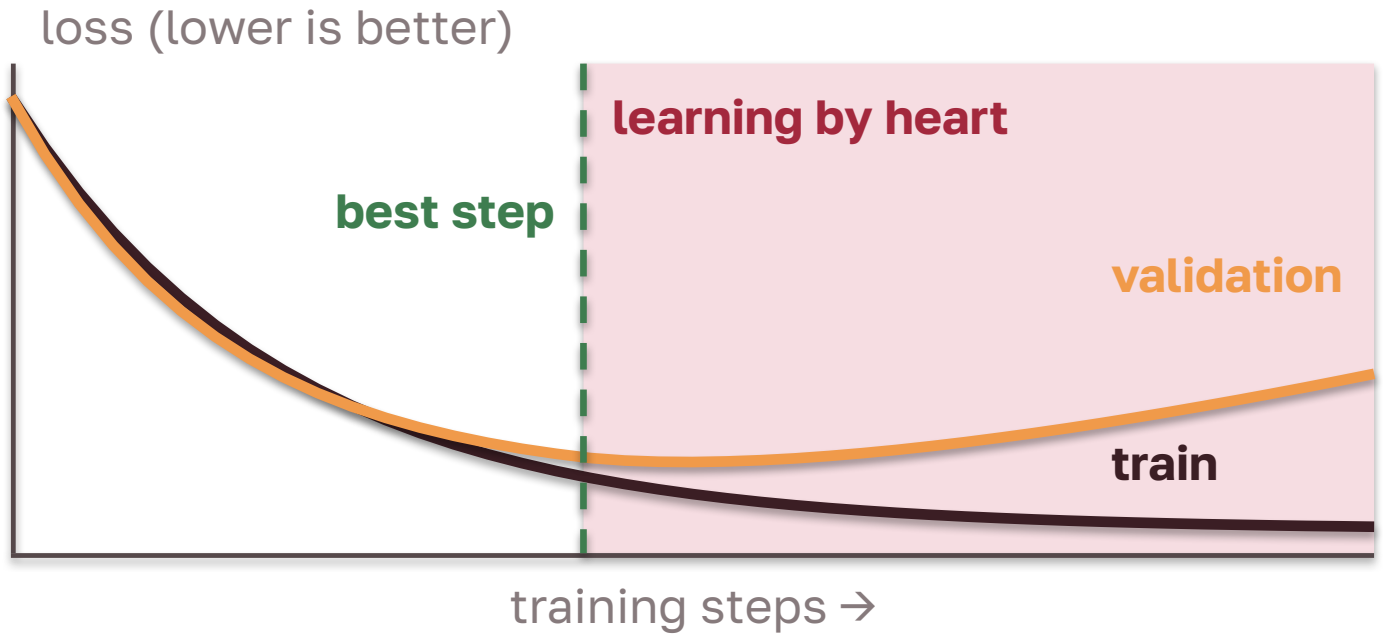
98.1 % right

on 7,351 hidden messages

Do not trust one batch: look at the trend and the test.

Train loss vs validation loss

Train loss = mistakes on the examples it learns from. Validation loss = mistakes on examples kept apart. Curves: drawn to show the idea.



NEW WORD

overfitting = learning by heart

Our tiny Kazakh GPT, validation on 300 hidden articles:

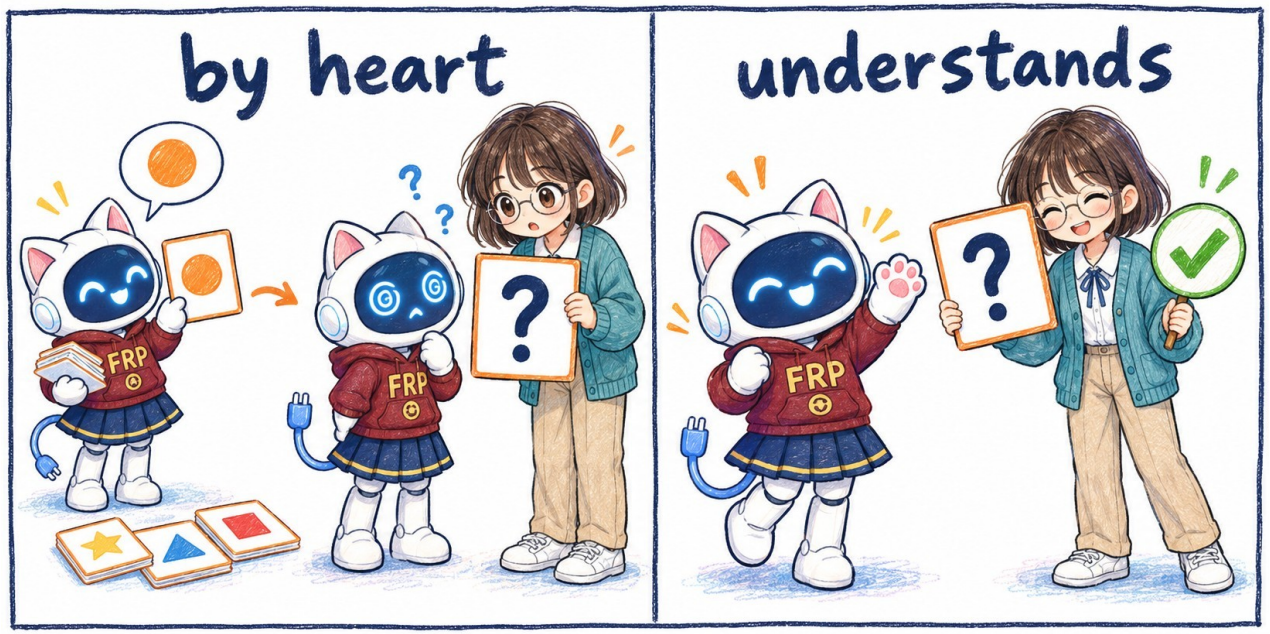
1.14 → 0.88

still going down at the end: no overfitting yet

Validation loss goes up = it learns by heart.

Stop at the best step

A model that sees the same few examples too often remembers the answers, not the rule. Then it fails on new questions.



NEW WORD

early stopping = keep the best step

1. Check the validation loss often.
2. Save the knobs at each check.
3. Keep the save with the lowest validation loss.

Keep the step that is best on hidden data.

Training needs 16 bytes for every knob

Full training keeps the knob, its change and 3 helper numbers: about 16 bytes per knob (just storing it needs 2). One of our GPUs (V100) has 32 GB.

our tiny GPT
10.9 million knobs



0.17 GB
1 GPU

Qwen3-0.6B
our school run



9.5 GB
1 GPU

an 8B model
8 billion knobs



128 GB
4 GPUs

Qwen3.8-27B
27 billion knobs



432 GB
14 GPUs

To use a model: 2 bytes per knob.

To train it: 16 bytes, 8 times more.

The model's working notes during training come on top.

Each square = one V100 GPU, 32 GB. Orange = fits on one GPU (we trained these).

16 bytes per knob: a big model needs many GPUs.

How much text for each knob?

Chinchilla (DeepMind, 2022): for a fixed budget, give each knob about 20 tokens of text. Double the knobs, double the text.

18

letters per knob

our tiny GPT (197 million letters)

≈ 20

tokens per knob

the Chinchilla rule (2022)

≈ 39

tokens per knob

Llama 3 405B (15.6 trillion)

≈ 1,900

tokens per knob

Llama 3.1 8B (about 15 trillion)

Llama 3.1 405B: 30.84 million GPU-hours ≈ 3,500 years on one GPU. It ran on up to 16,000 GPUs and stopped 466 times in 54 days.

Small models read far more: costly once, cheap to use.

PART B: GOING TO SCHOOL

Stage 2: it goes to school

Fine-tuning: copy good example answers.

How do we know a model is better? A benchmark

Before school: how do we see that a model got better? We give it an exam.

A benchmark = a big exam for models

the same questions for every model

the same rules

one score: % right answers

so everyone can compare

A real KazMMLU question

Хан Тәңірі шыңының биіктігі (м)?

How high is Khan Tengri peak, in metres?

A 4973 B 6995 C 4506 D 4622

Answer key: B

KazMMLU, 2025: 23,000 questions

A benchmark is like your Day 5 test set: big, and the same for everyone.

QWEN3-0.6B-BASE: ONLY STAGE 1, ASKED IN CHAT FORMAT

A reader is not an assistant

Qwen3-0.6B-Base: a real model (0.6 billion knobs) after stage 1 only. We asked it in a chat.

You What is the capital of Kazakhstan?

Model What is the capital of China? ⚇ ⚇ ⚇ ⚇ ⚇ ⚇ ⚇ ⚇ ⚇ ...

You What does a programmer do?

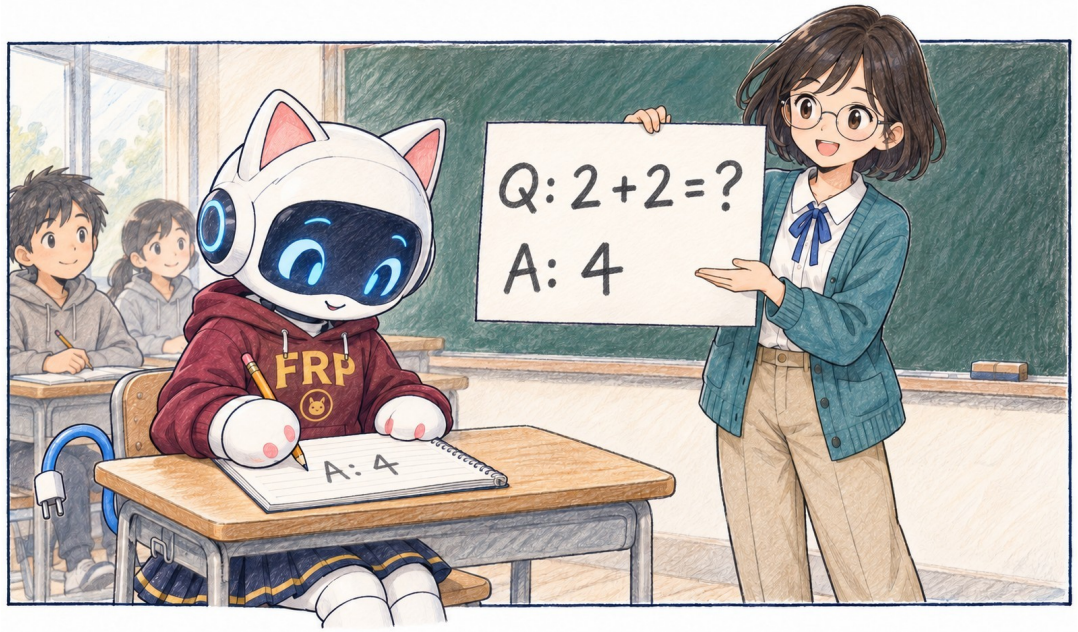
Model What is the difference between a programmer and ...

As plain text (not a chat) it often answers: 18 of 20 questions. It lacks the chat form.

> **In a chat it does not answer. It only continues the text.**

School = example answers

Stage 2, school: the model copies good example answers. We did it with Qwen3-0.6B-Base.



4,000

examples: a question + a good answer (SmolTalk, a free dataset)

20 minutes

of training on 1 GPU (V100)

It learns the form: you ask, it answers, it stops.

More training of a ready model on example answers is called fine-tuning.

ACTIVITY: VOTE A OR B, 4 MINUTES

Before or after school?

2 real answers per question: one before school, one after. Hands up: which is after, A or B?

1. What is the capital of Kazakhstan?

A What is the capital of China? ⚇ ⚇ ⚇ ⚇ ⚇ ⚇ ⚇ ⚇ ⚇ ⚇ ...

B The capital of Kazakhstan is Astana. It is a ...

2. Write one sentence that uses the word 'curious'.

A The curious child's eyes sparkled with wonder ...

B Write one sentence that uses the word ...

3. How do plants make food?

A How do plants make food? ⚇ / ...

B Plants make food through a process called ...

After school the model answers the question, and then it stops.

20 minutes of school

We asked 20 new questions in a chat, before and after school (4,000 examples, 20 minutes).



stops by itself, of 20

1 → 14

(not talking forever)

questions answered in the chat, of 20 (higher = better)

InstructGPT (OpenAI, 2022): people liked a small 1.3-billion-knob model after school more than the 175-billion GPT-3 without school.

> **School is quick: 4,000 examples, 20 minutes. Reading took much, much more.**

After school. But is it true?

We asked our model after school a simple question. Read its answer: is every part right?

You: What is the capital of Kazakhstan?

Model: “The capital of Kazakhstan is Astana. It is a city located in the southern part of the country, **near the Caspian Sea.**”

The fact: Astana is in the north of Kazakhstan, far from the Caspian Sea.

> **School teaches HOW to answer, not WHAT is true.**

A dataset = many examples

All the examples together = a dataset. Real rows from our 3 training runs, one per stage.

Reading

plain text:

“Төрежан Мәндібай - 1937 жылы 18 наурызда …”

60 million letters

School

question + answer:

“average of 21, 35, 12, 45, 67?” → “… = 36”

4,000 pairs

Practice

question + a checker program:

“how many l in flexible?” → is it 2?

6,000 questions

Each stage needs a different kind of data.

Keep some data hidden

In every run we kept some data apart. The model never trained on it; we tested on it.

Reading

learn: 60 million letters

hidden test: 300 articles

School

learn: 4,000 chats

hidden test: 20 new questions

Practice

learn: 6,000 questions

hidden test: 300 new words

Test on data the model never saw. Else it can learn the answers by heart.

Where to get data for your project

Take a ready dataset (Hugging Face = a free website of datasets), or make your own.

Take: Hugging Face

KSC2 (ISSAI): 1,128 hours of Kazakh speech

Food (ISSAI): 16,402 photos, 42 kinds

Kazakh Wikipedia: 245,618 articles

Make: your own

write, record or take photos

give every example its answer

check: licence (allowed?), consent (did people say yes?), junk

Small, clean and checked data beats big and dirty data.

A new language: ISSAI's Kazakh models

Reading and school can also teach a new language. ISSAI (our institute) did it twice.



KAZ-LLM (2024)

built on Llama 3.1: 8 and 70 billion knobs

150 billion tokens: Kazakh, Russian, English, Turkish

Qwen3.5-9B-Kazakh (2026)

built on Qwen3.5-9B: 9 billion knobs

How we made it: the next 3 slides

Start from a strong model. Teach it what is missing.

Step 1: new Kazakh pieces for the tokenizer

Qwen learned mostly English and Chinese, so it cuts Kazakh words into many small pieces (tokens). ISSAI added 16,000 new Kazakh pieces.

before Қ | а | зақ | стан → after Қазақстан

NEW WORD

tokenizer = the tool that cuts text into tokens

before м | ем | лек | ет | тік → after мемлекеттік

Tokens per Kazakh word: **3.76 → 2.13** (43 % fewer tokens)

English for comparison: 1.61 per word

> **Fewer pieces: faster, cheaper, and more Kazakh fits in its memory.**

Step 2: read Kazakh, and do not forget English

The 16,000 new pieces are new knobs. So the model read more text: continued pretraining.

New pieces start smart

a new piece starts as the average of the old pieces it replaces:

Қ + а + зақ + стан

Continued reading

15.9 billion tokens

half Kazakh, half English

on 16 big GPUs (H100)

Why English too?

without it, the model forgets English

English stays almost as good

Teach the new language, and keep the old skills.

Step 3: copy the chat skill from Qwen

Qwen has a “base” version (after reading) and a “chat” version (after school).

Qwen chat − **Qwen base** = **the chat skill**

Kazakh base + **the chat skill** = **Kazakh chat**

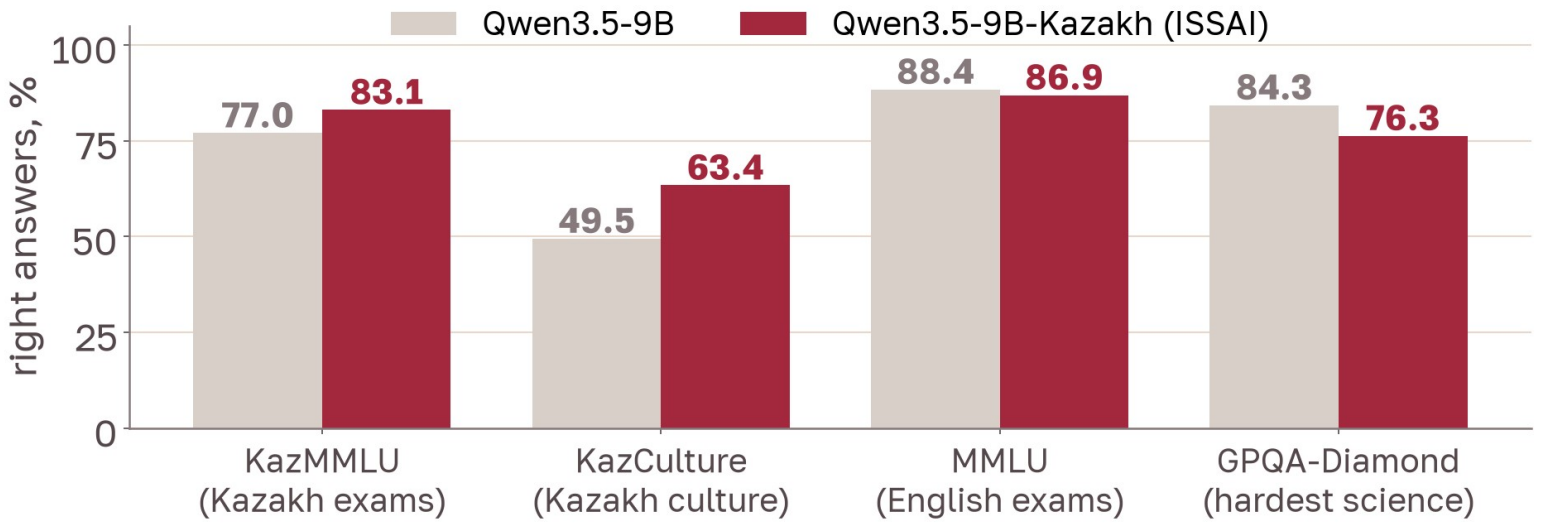
How? knob by knob: chat value − base value, for all 9 billion knobs.

Why maths? Good Kazakh chat examples are rare. Qwen's school already cost a lot.

> **No Kazakh school needed: reuse what Qwen's school already taught.**

Did it work? We measured on benchmarks

Grey = Qwen3.5-9B, red = our Qwen3.5-9B-Kazakh, on the same 4 benchmarks. Bars = % right answers (higher = better).



change, % points

↑ KazMMLU +6.1

↑ KazCulture +13.9

↓ MMLU -1.5

↓ GPQA -8.0

Kazakh went up a lot. English went a little lower. The hardest science went lower.

3 ways to teach a ready model

School starts from a model that already read a lot. Which of its knobs may change? Orange = knobs that learn. Dark = frozen: they stay the same.



1. Change all knobs

full fine-tuning

all knobs learn

our school run: 20.5 min

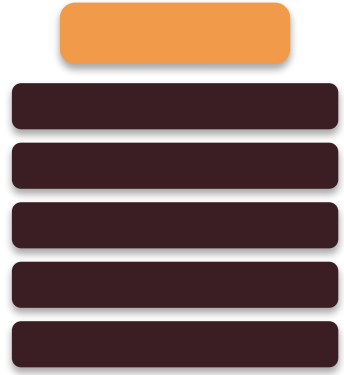


2. Add a small add-on

LoRA

old knobs frozen

GPT-3: 10,000× fewer to train



3. Freeze + new head

a new last layer

only the new head learns

ants vs bees: 240 photos

Three ways: all knobs, a small add-on, or a new head.

LoRA: a 35 MB add-on for a 350 GB model

LoRA freezes the big model and trains a few new knobs next to each layer. Test model: GPT-3, 175 billion knobs (2021).

350 GB
the frozen model

35 MB
the add-on



10,000×
fewer knobs to train
as good as changing all knobs

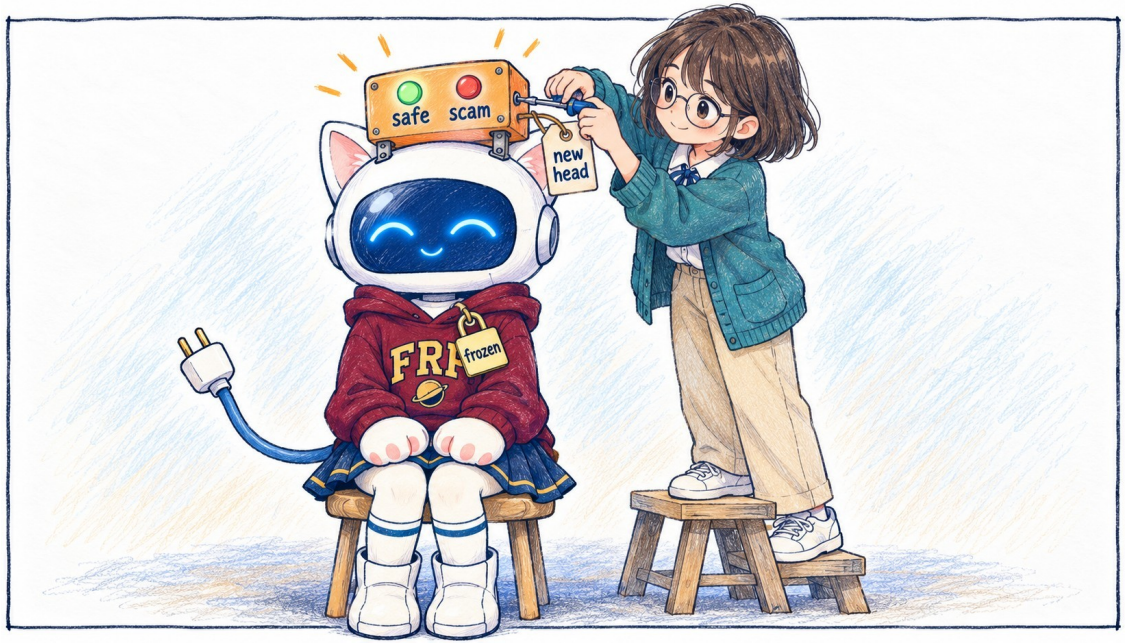
354 GB
for 100 skills
1 model + 100 add-ons, not 35 TB

1 GPU
QLoRA (2023)
a 65B model on one 48 GB GPU, not 780+ GB

LoRA trains a tiny add-on: one big model, many small skills.

Freeze the model, add a new head

A ready model already turns a photo or a text into useful numbers. A new last layer turns these numbers into YOUR answer.



NEW WORD

head = the last layer, it gives the answer

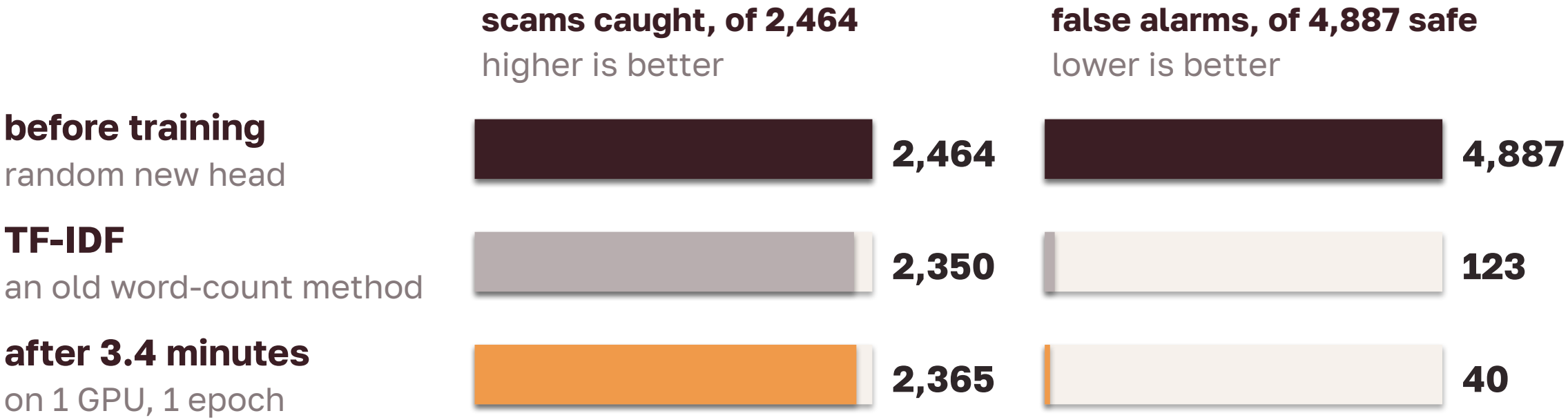
Ants or bees? 240 photos (PyTorch guide)

all knobs learn	**94.8 % right, 37 s**
frozen + new head	**94.8 % right, 28 s**

If the old model already sees well, a new head is enough.

The scam-detector: a new head, 3.4 minutes

A ready language model (xlm-roberta-base) + a new head: safe or scam. Test: 7,351 hidden messages, 2,464 scams and 4,887 safe ones.



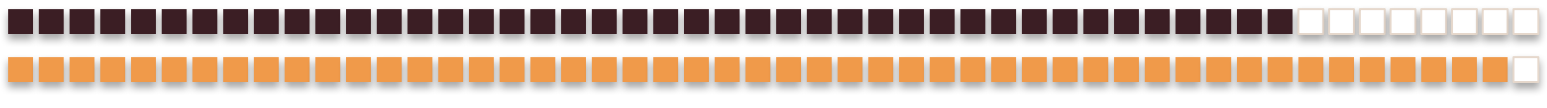
3.4 minutes: false alarms 4,887 → 40, scams caught 2,365.

Local data: 42 → 49 of 50 scams caught

The first model learned from messages that were 99 % English. Second training: + 272 local messages (75 Kazakh scams, 197 AI-written), each repeated 10 times.

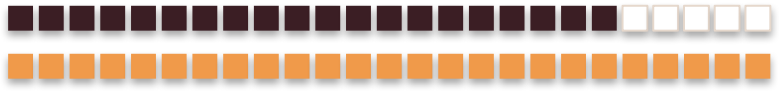
scams caught of 50

42 → 49



Kazakh scams of 25

20 → 25



false alarms of 50 safe

1 → 2



lower is better

Exam: 100 real local messages (50 Russian, 50 Kazakh). Each square = one message.

272 local messages helped more than 66,000 foreign ones.

Good school beats size: 1.3B beat 175B

OpenAI, 2022: people compared the answers of two models to the same questions and chose the one they liked more.

GPT-3

175 billion knobs

reading only

InstructGPT

1.3 billion knobs

people liked it more

1. School: 13,000 questions with answers by about 40 people

2. Ranking: people ranked answers (33,000 questions)

3. Practice: with a score (31,000 questions, Part C)

about 2 % of the reading compute for school + practice

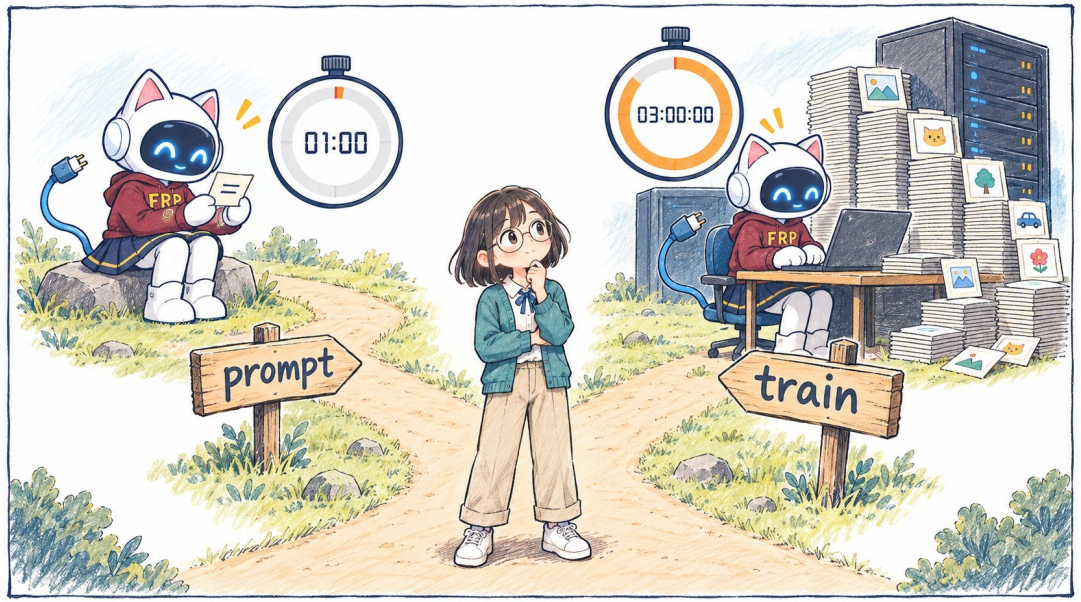
Good examples beat 100 times more knobs.

Write a prompt, or train?

Two ways to make a ready model do your task. Aru must choose one for her project.

prompt

- minutes, no GPU
- change it any time
- the school-video, quiz-bank, university-guide projects



train

- many examples with answers
- a GPU + a hidden test
- the scam-detector, the Tetris project

Try a prompt first. Train when a prompt is not enough.

Should I fine-tune? Ask 2 questions

A decision for your project. Labelled example = an example with the right answer (a message + "scam" or "safe").

Does a good prompt already work?
test it on real examples

→ **no**

Do you have many labelled examples?
each with the right answer

→ **yes**

Fine-tune
a new head or LoRA (the scam-detector)

↓ **yes**

Use the prompt
the quiz-bank project

↓ **no**

Collect data first
or no training: the song-recommender

Always: keep a test set apart before you train.

Prompt first, data second, training third.

Keep the test apart, check for leaks

Day 5: test only on data the model never saw. The scam-detector split its 73,511 messages before training.



✓ Exact copies: 0

The mentors searched: no test text is in the training data.

? Near-copies: maybe

Kazakh test and training scams came from one list. The same scam with a new phone number is almost a copy.

Check for copies AND near-copies before you trust a score.

Break, 10 minutes

NEXT

C practice with a score: how models learn to think

D how this class trained

E how trainings differ

F your turn: you train a model

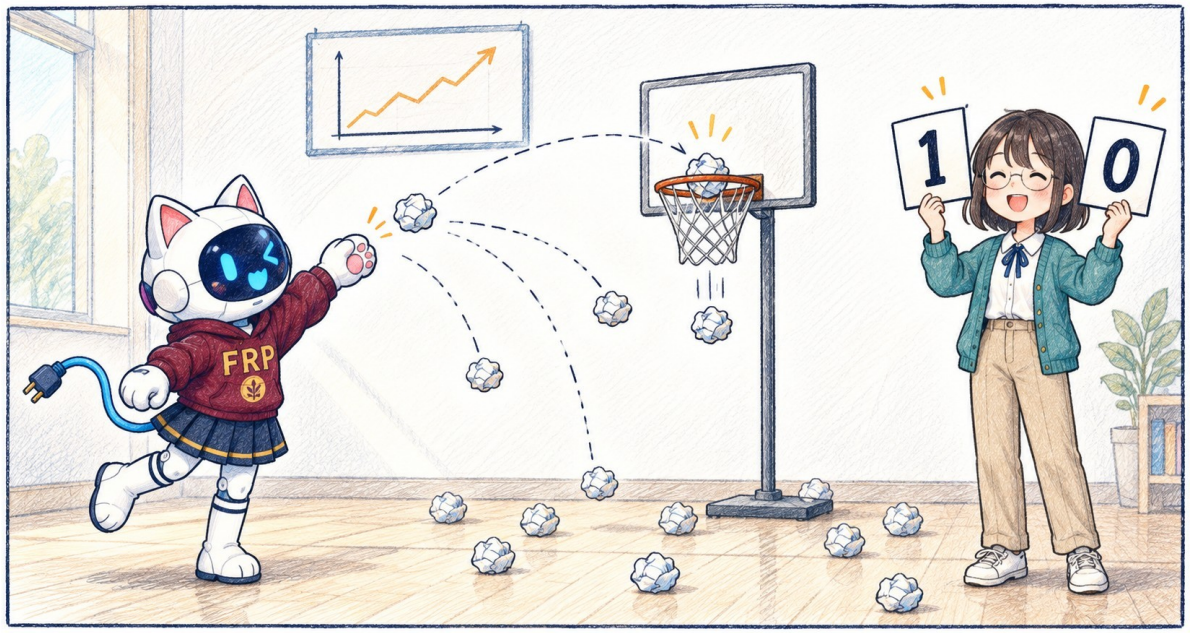
PART C: PRACTICE WITH A SCORE

Stage 3: it practises with a score

Reinforcement learning: nobody shows the answer.

School shows answers. Practice gives a score.

Stage 3. In practice nobody shows the model the answer: it gets only a score.



In the picture: the AI (robot cat) throws, Aru shows the score: **1** = a basket, **0** = a miss.

Nobody shows it the right throw.

Try → score → try again.

This is **reinforcement learning (RL)**.

Try, get a score, try again: you learn to throw a ball into a basket the same way.

Easy for you, hard for AI: count letters

Our model needs a practice task: easy to score, hard for the model. Try it yourself first.

How many “e” in **experience**?

experience = 1 piece (token) for Qwen: no letters inside.

You see 10 letters (answer: 4). The model sees only 1 piece, like 1 emoji.

Our practice task for the model: count one letter in a word.

ACTIVITY, 2 MINUTES: HANDS UP

You are the score: 1 or 0?

The rule: 1 point if the number is right. 0 if it is wrong. Real answers of our model. Count yourself, then give 1 or 0.

How many “r” in library? Model: **2**

How many “x” in ascending? Model: **1**

How many “h” in rhythms? Model: **2**

How many “p” in prepare? Model: **1**

The model gets only this number: 1 or 0. Nobody shows it how to count.

OUR PRACTICE METHOD: GRPO

Try 8 times, keep what works

One question: How many “r” in library? The model answers 8 times. The checker gives each answer 1 or 0 (an example).

try 1	try 2	try 3	try 4	try 5	try 6	try 7	try 8
2	**3**	**2**	**1**	**2**	**3**	**0**	**2**
✓ 1	✗ 0	✓ 1	✗ 0	✓ 1	✗ 0	✗ 0	✓ 1

✓ right answers

→ the model makes answers like these MORE often

✗ wrong answers

→ LESS often

Then the next question, thousands of times: this is practice with a score.

Our model after 4 minutes of practice

Our real run, 14 Oct: Qwen3-0.6B practised letter counting with GRPO for 258 seconds on one V100. Test: 300 new words it never saw (higher = better).

36.3 → 50.7 %

right answers

before → after practice

0 → 49 %

answers that spell the word

nobody taught it to spell

Spelling = **l, i, b, r, a, r, y → two “r”**: a method it found by itself, only from the score.

Only a score, no examples: the model got better and found its own method.

A REAL ANSWER FROM PRACTICE

Right answer, wrong reasoning

The checker looks only at the last number. Look at the whole answer.

How many “c” in “desired”?

Model: d, e, s, **t**, e, d … Answer: 0

Score: 1 point (0 is right) but “desired” has no “t”: the spelling is wrong.

> **The score checks only the last line. Making a good score is the hard part.**

You get what you reward

OpenAI (2016): an AI played CoastRunners, a boat race game. The score: points for hitting targets on the way.

+20 %

higher score than human players

But it never finished the race. It turned in circles, on fire, hitting the same 3 targets.



NEW WORD

reward hacking = a high score without doing the real task

The AI does what the score rewards, not what you meant.

This is how reasoning models are made

DeepSeek-R1-Zero (2025) practised maths with only a right/wrong score, like our model. Test: AIME 2024, a hard maths contest (higher = better).

15.6 → 77.9 %

right answers

before → after practice

“wait, let me check”

it learned by itself to think longer and check its work

DeepSeek-R1, Nature 2025 / arXiv 2501.12948 v2

The same idea as our letter counting, at a giant size.

Games and language models: both practise

The same idea, two starting points. The score can be game points or a checker's 1 or 0.

Game AI

Starts from: **zero**

Score: **game points**

Examples: Atari games (2015), our Tetris project

Language model

Starts from: **it can already write**

Score: **a checker, 1 or 0**

Examples: our letter counting, R1-Zero



A game AI learns to play from zero. A language model only sharpens what it can already do.

Who gives the score: a checker or people?

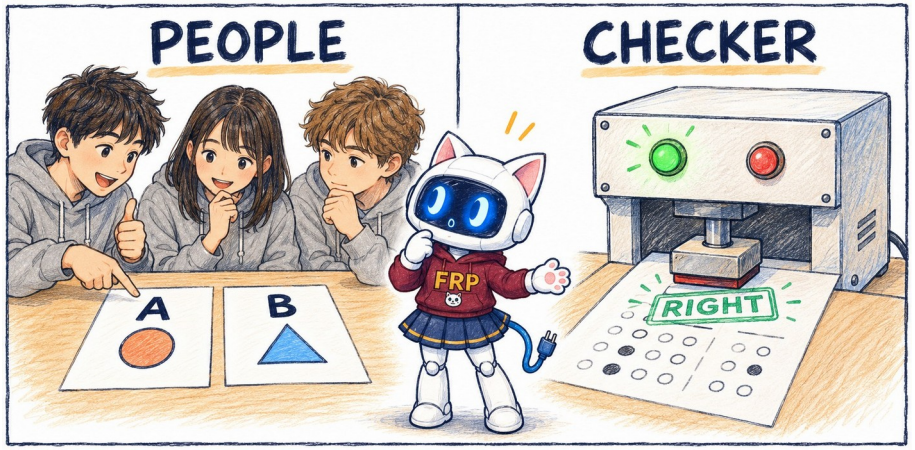
Every practice needs a score. A checker tests the answer. For “helpful” or “polite” there is no right answer, so people choose.

A checker

right = 1, wrong = 0

For: maths, code tests, letter counting

Hard to fool



People

people choose the better answer

For: helpful, polite

Can be fooled

Use a checker when the task has a right answer. Use people when it does not.

Practice is cheap next to reading

DeepSeek's own numbers: the cost of the GPU time (rented at $2 per GPU-hour) for the final runs.

$5.6 million

reading: the base model DeepSeek-V3

$294,000

practice + school: DeepSeek-R1 (about 5 %)

OpenAI, InstructGPT: school + practice = under 2 % of reading.

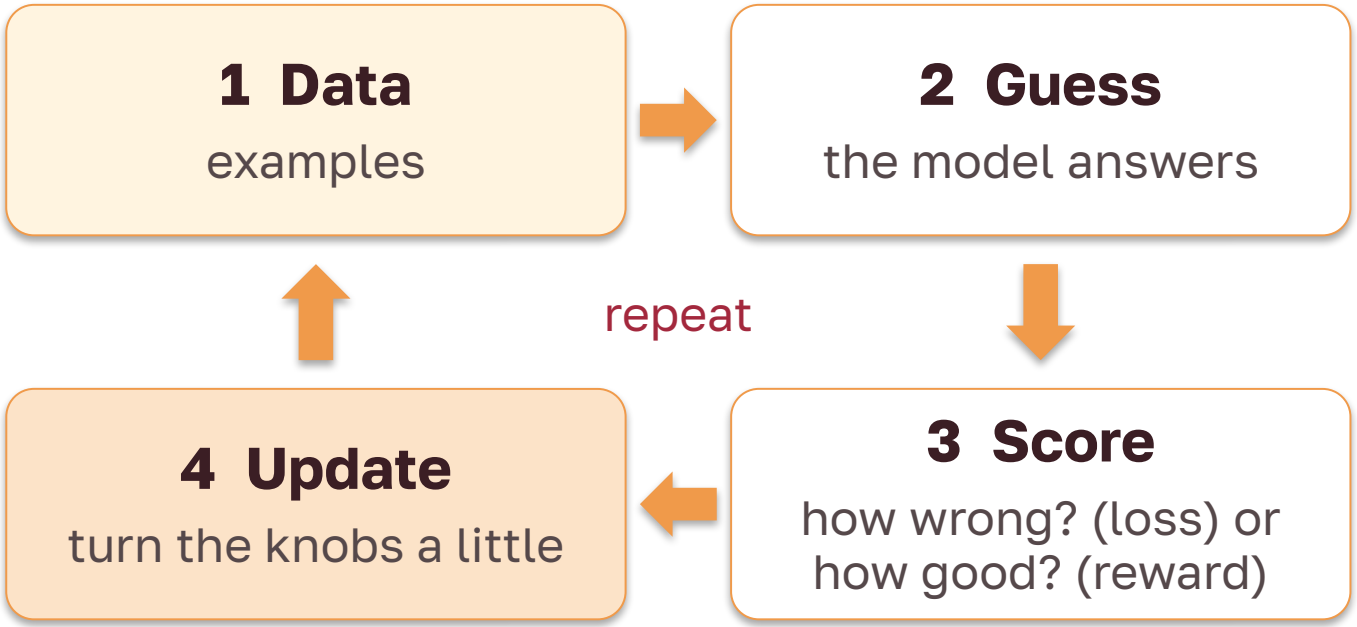
> **Reading costs almost everything. Practice is a small, cheap last step.**

How this class trained

Your projects this week: school with answers, practice with a score, a ready model, a simple rule.

Every training is one loop

Parts A–C showed our own runs. Now your class. We draw every project with the same loop, repeated thousands of times.



School (fine-tuning)
the data has the right answer

Practice (RL)
no answers, only a score

No training
a ready model, no steps 3–4

Training = guess, score, update, repeat.

Scam-detector: a ready model + a new head

The scam-detector project: a model reads a message and says “scam” or “safe”. The team did not start from zero: they took a ready model.

Message
“Card blocked!”

→

XLM-R
ready reader

→

New head
random at start

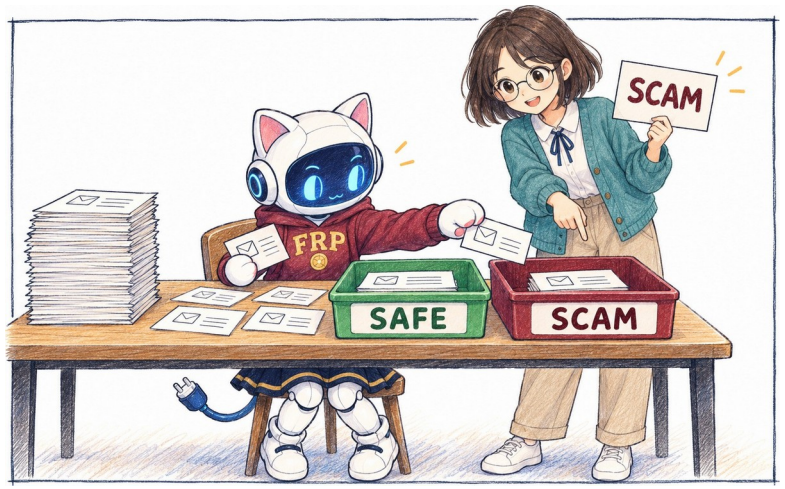
Data: 73,511 public messages (99% English)



7,351 for the test

NEW WORD

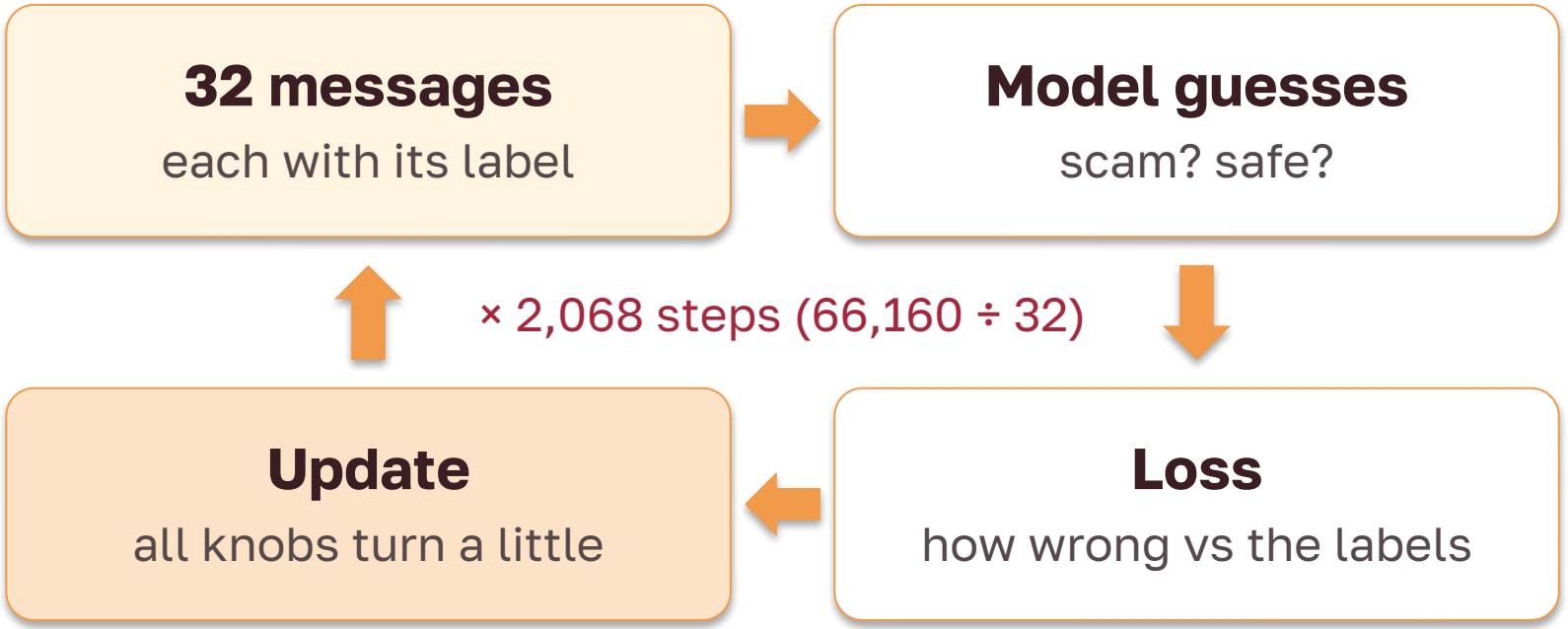
head = the last layer: it gives the answer



Start from a ready model. Teach only the new job.

Its loop: school with the right answers

Fine-tuning with labels: every training message comes with its right answer (“scam” or “safe”). The model learns from 32 messages at a time.



1 epoch

= every training message seen once

3.4 min

on 1 V100 GPU

Show the answer, measure the mistake, turn the knobs.

3.4 minutes: random head → 98.1% right

Both tested on the same 7,351 kept-apart messages: 2,464 scams and 4,887 safe ones. Higher "right" is better.

Before: random head

33.5%

says "scam" to every message
caught 2,464 of 2,464 scams
false alarms: 4,887 of 4,887 safe

After 3.4 minutes

98.1%

right on 98.1% of 7,351
caught 2,365 of 2,464 scams
false alarms: 40 of 4,887 safe

Old method TF-IDF (counts words, seconds on a CPU): **caught 2,350, false alarms 123**

Minutes of school turned random guessing into 98.1%.

Second training: add local messages

The first data was 99% English. The team added 75 local Kazakh scams + 197 AI-written messages (each × 10) and trained again: 3.5 min. Exam: 100 real local messages (50 Russian, 50 Kazakh).

	before		after	
Scams caught higher is better	42	➡	49	of 50
Kazakh scams caught higher is better	20	➡	25	of 25
False alarms lower is better	1	➡	2	of 50 safe

Careful

0 test texts were in training. But test and training Kazakh scams came from one list: near-copies are possible.

Local examples helped: 42 → 49 of 50 scams caught.

Tetris: practice with a score

The Tetris project: a program learns to play Tetris by itself. Nobody shows it the right move. It only gets a score: the lines it clears.

Board
the blocks now

→

4 numbers
describe it

→

Network
4→64→64→1
scores each place

↓

Pick the best
or a random one

←

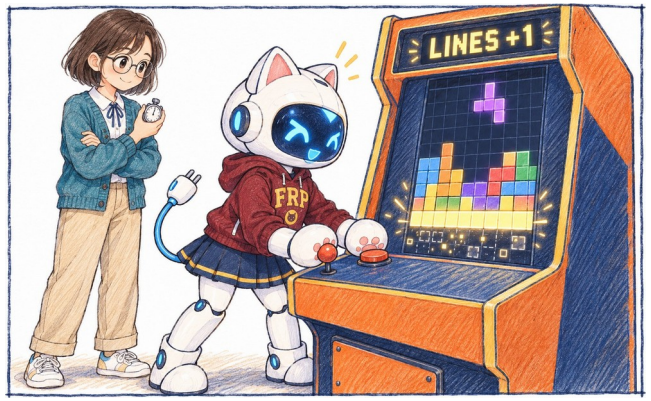
Reward
= lines cleared

←

Learn
update the knobs

↑

repeat for every piece



NEW WORD
reward = the score after a move

NEW WORD
exploring = trying a random move

No answers, only a score: it learns by trying.

1,064 games: the real learning curve

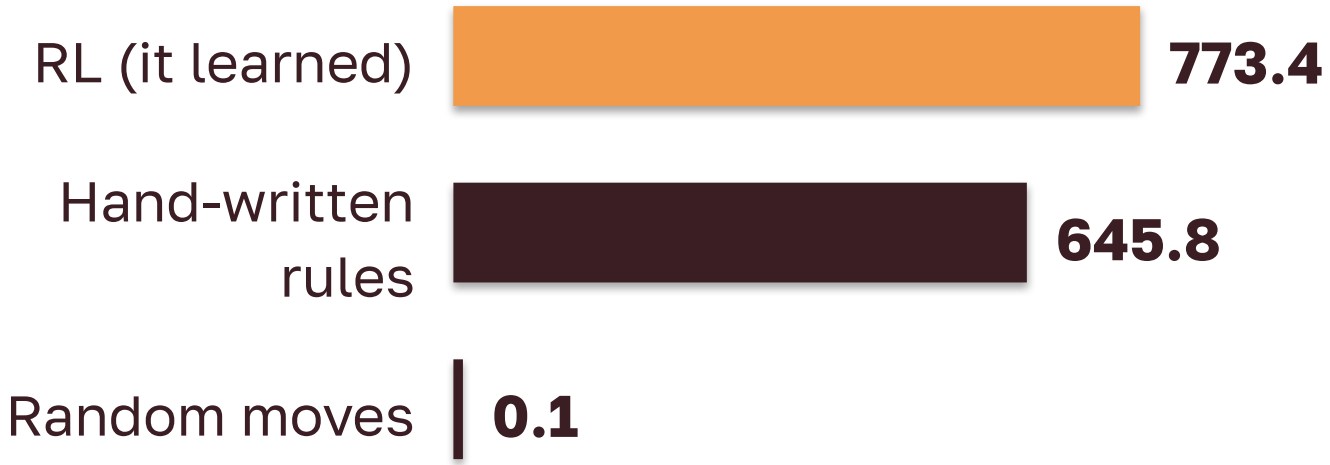
From the project's own training log (27 Oct, 1 V100 GPU, 23.1 min). Each bar = lines cleared per game, average of 28 games in a row. Higher is better.



900 games looked hopeless. Then it learned in minutes.

Fair test: RL wins, then hits the limit

20 new games. Every player gets the same blocks in the same order (same seeds). At most 2,000 pieces per game. Lines cleared per game, average: higher is better.



spread (±): RL 109.3, rules 200.6 lines

19 of 20

RL games stopped at the 2,000-piece limit.

The game stopped, not the player. The test is now too easy.

When games hit the limit, the test stops measuring.

Song-recommender: no training at all

The song-recommender project: find new songs that a friend will like. It uses a ready model as it is: no knob changes.

Song
3 pieces of 10 s

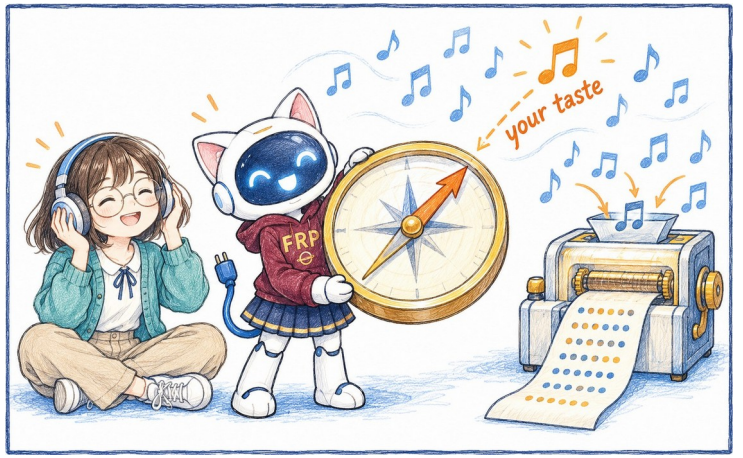
→

CLAP
a ready sound model

→

512 numbers
for each song

Steps 3–4 of the loop (score, update) **never run.**



NEW WORD

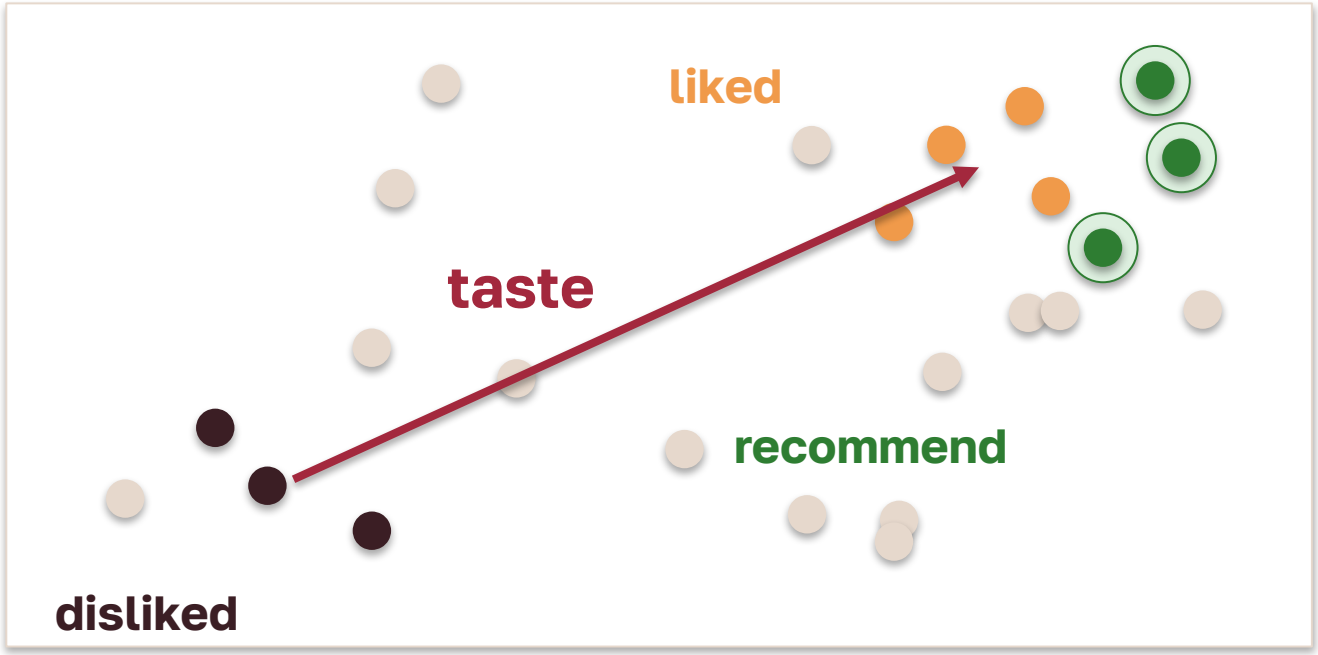
embedding = numbers that describe a thing

47 s to embed all 76 songs, on a GPU

A ready model turns songs into numbers. No training.

Taste = liked – disliked

Each song is a point (here 2 of its 512 numbers). Taste = average of liked songs minus average of disliked songs. Recommend the songs nearest to the taste.



6 of 9

hidden favourite songs found

in a friend test. Random picking finds about 0.05 per friend.

Simple maths on ready numbers found 6 of 9 songs.

Solar-forecast: the simple rule won

The solar-forecast project: predict tomorrow's energy from a solar plant. Tiny models learned on 30 days of June. Test: 25 normal days of July.

Error per day in July, kWh (energy) lower is better





September (30 days): the rule missed only **513 kWh (6.8%)**

30 days were too few. An honest “no” is a result.

Three projects with no training

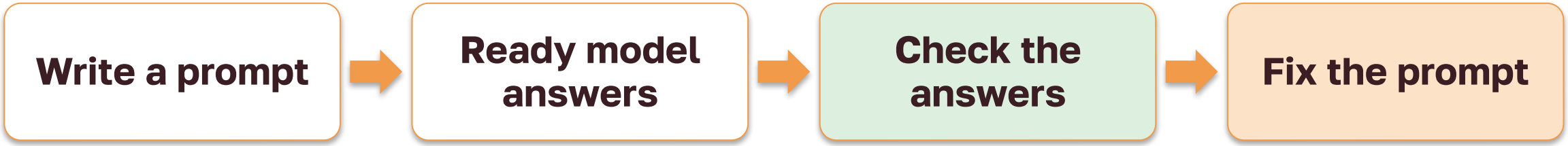
These projects use ready models. Their work was data, prompts and checks. The loop is still there, but a person changes the prompt.

school-video
prompts + an AI checker

quiz-bank
questions from the class model

university-guide
an agent reads web pages

Their loop: a person, not the computer, makes the update



Change the prompt and the data before you train.

Your class side by side

7 projects, 4 kinds of work. Only 3 of them trained a model; only 2 beat a simple way.

project	kind	data	time, hardware	what changed	result
scam-detector	fine-tuning	66,160 messages	3.4 min, 1 V100	all knobs + new head	**98.1% right**
Tetris	RL (practice)	1,064 own games	23.1 min, 1 V100	a new small network	**773.4 vs 645.8 lines**
song-recommender	none	76 songs	47 s, 1 GPU	nothing	**6 of 9 found**
solar-forecast	tiny models	30 days	–	tiny models	**simple rule won**
3 others	none	prompts, checks	–	nothing	–

Train only when a ready model or a simple rule fails.

How trainings differ

Six ways to teach a model: from 16,000 GPUs for months to no training at all.

Six ways to teach a model: what changes

From the biggest change to none. Weights = the model's knobs (Part A).

Way	What changes	Data it needs	Example
Pretraining	all weights, from random	trillions of words	our tiny GPT (7 min)
Full fine-tune	all weights, a little	thousands of examples	the scam detector
LoRA	an add-on, from 0.01 %	labelled examples	not used this week
Freeze + new head	only a new last layer	hundreds of examples	ants/bees: 240 photos
RL	the weights, by a score	questions + a score	the Tetris project
No training	nothing	a prompt or examples	song recommender

Only pretraining starts from random. All other ways start from a ready model.

Six ways to teach a model: what it costs

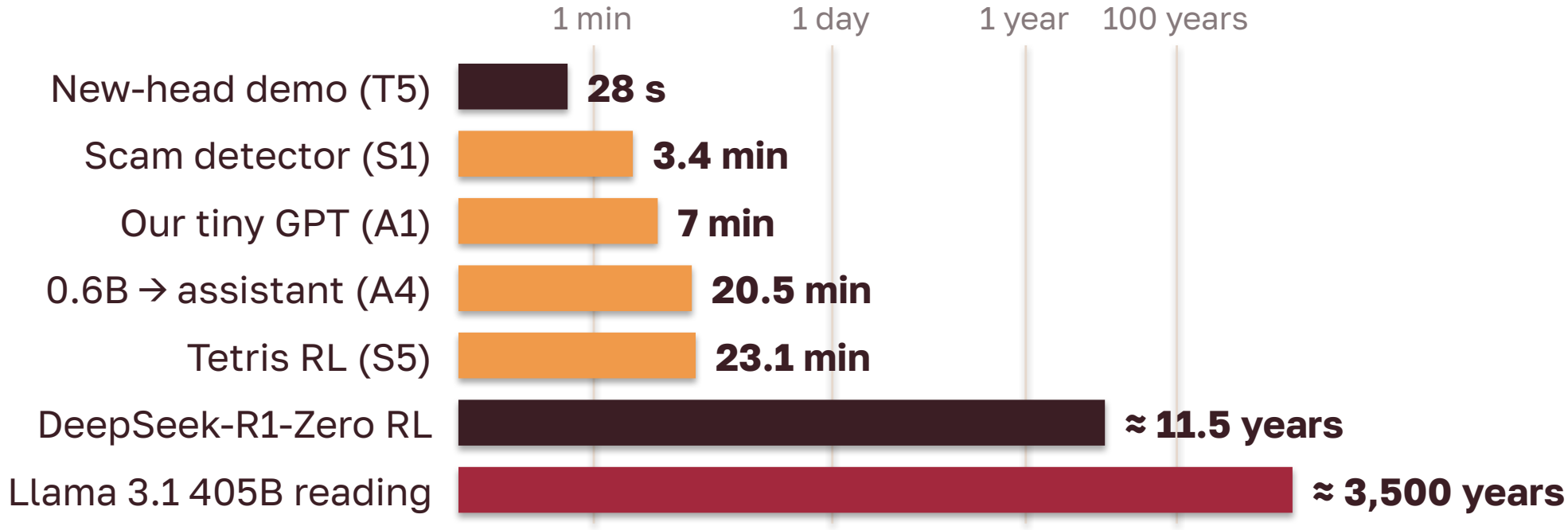
Training needs about 16 bytes of GPU memory per weight. Only using it needs 2 (ZeRO paper).

Way	GPU memory to train	Hardware: a real run
Pretraining	16 bytes per weight, and more	Llama 3 405B: up to 16,000 GPUs
Full fine-tune	16 bytes per weight: 1.5B → 24 GB	scam detector: 1 GPU, 3.4 min
LoRA	175B model: 1.2 TB → 350 GB	saved file: 350 GB → 35 MB
QLoRA	65B, squeezed: 780+ GB → 48 GB	one 48 GB GPU, 24 hours
Freeze + new head	learns only the last layer	a CPU can be enough
RL	like fine-tune, + many tries	R1-Zero: 512 GPUs, 198 hours
No training	only to use it: 2 bytes per weight	song recommender: 76 songs, 47 s

Full training needs 8 times the memory of using a model. LoRA and a new head need much less.

From seconds to 3,500 years

Training time if it ran on ONE GPU (of different kinds). Log scale: each grid line is many times more. Shorter = cheaper. Orange = runs in this class, on one V100.



Our runs take minutes. A big model's reading takes thousands of GPU-years.

Big runs break

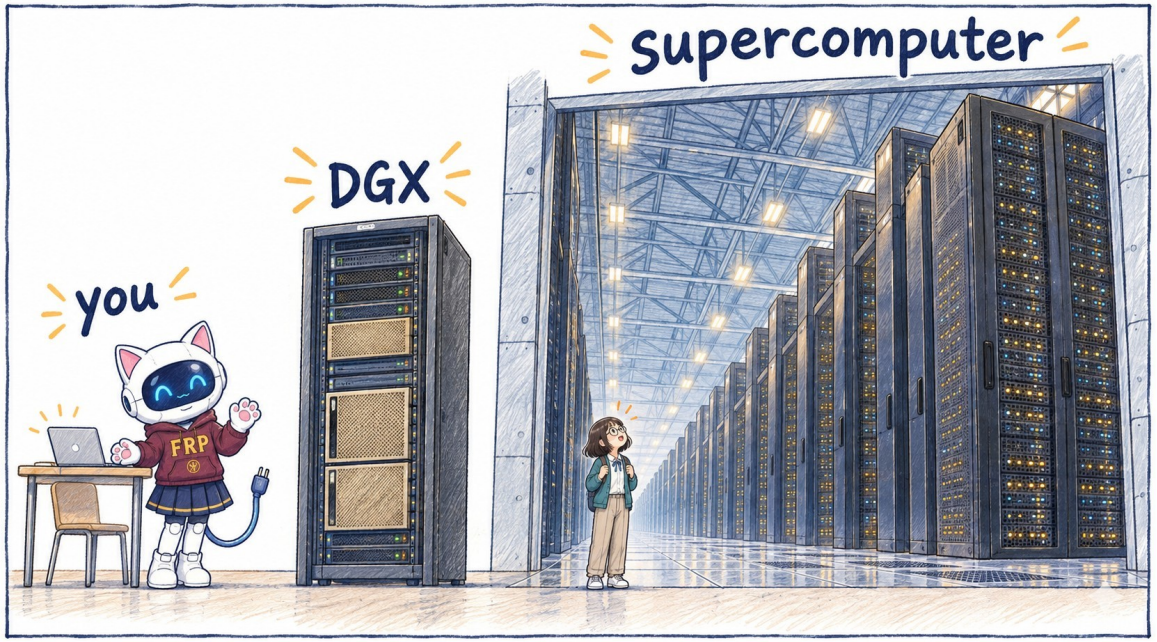
To finish in months, Meta trained Llama 3 405B on up to 16,000 GPUs at once. With so many machines, something breaks all the time.

466 stops

of the training job in 54 days

78 %

of the unexpected stops: broken (or maybe broken) hardware



At big scale, training is also a fight with broken machines.

Sometimes the best training is none

Ready models turn photos, sounds or text into numbers. Compare them: no training.

CLIP: photos

Matched a trained ImageNet model, using **0 of its 1.28 million** labelled photos

CLAP: sounds

91.0 % right on 50 kinds of sounds, with no training on them

Song recommender

This class: CLAP numbers per song, no training. Found **6 of 9** hidden favourites.

NEW WORD

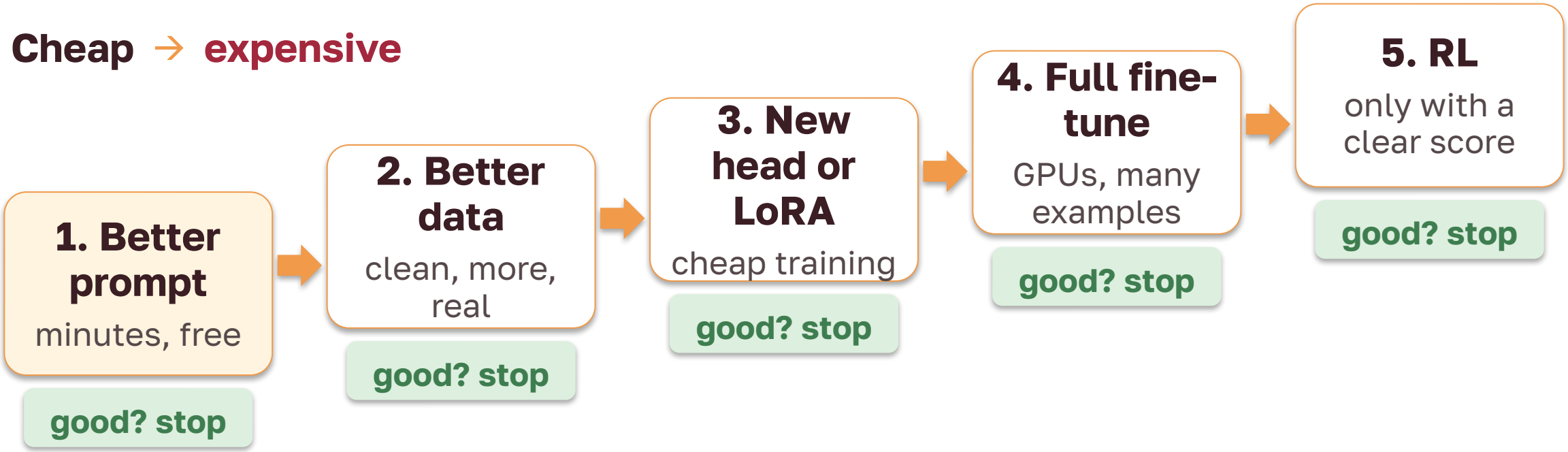
distillation = a big model teaches a small one

DistilBERT: 40 % smaller, keeps 97 % of BERT's skill, 60 % faster.

Before you train, check if a ready model already does the job.

Should I train?

Go left to right. After each step, test on your kept-apart test set. Good enough? Stop there.



Try the cheap step first. Train only when the cheaper steps are not good enough.

3 common mistakes

We saw all three this week: in this lecture and in Day 5 (testing).

1. Test data in training

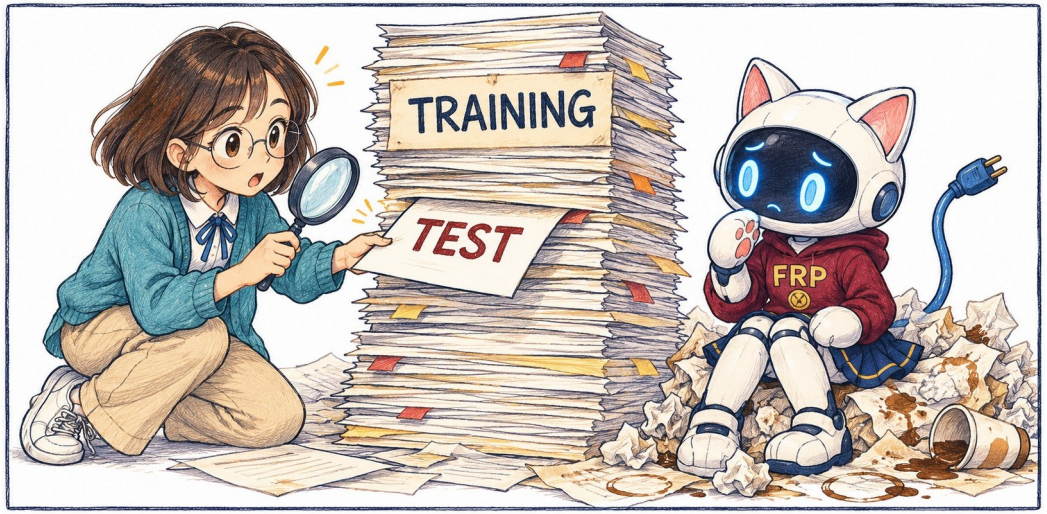
a leak (Day 5): a great score that means nothing

2. A score that rewards the wrong thing

the boat in circles; our tiny model's "cɵɜ cɵɜ cɵɜ"

3. Too little or dirty data

dirty text → our model wrote junk in 37 of 50 texts



Keep the test apart, check what the score rewards, and clean your data.

Where your jobs run

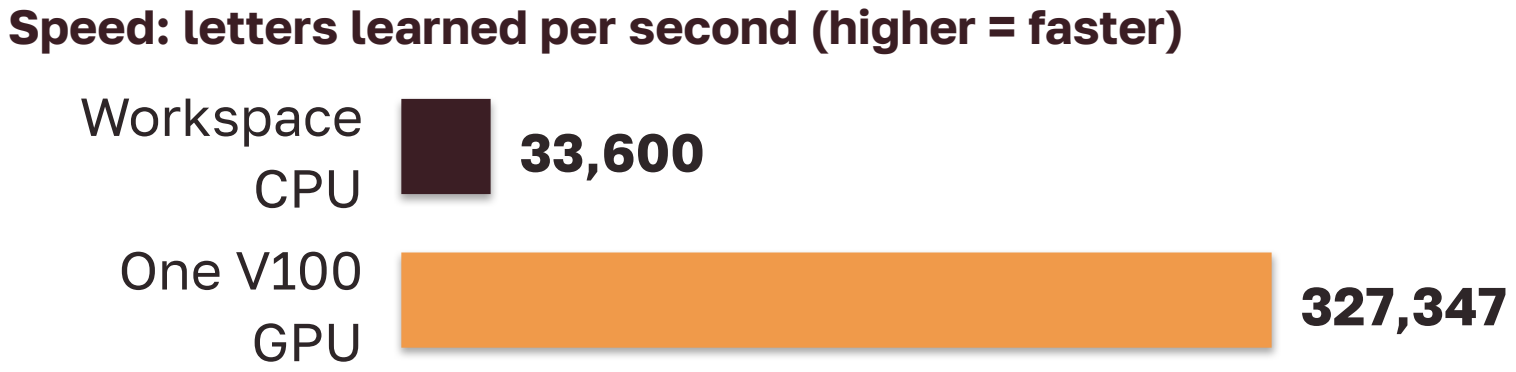
16 GPUs, one queue, a whole class: how we share them.

One server, 16 GPUs

The class server has 16 Tesla V100 GPUs (32 GB memory each). Your workspace has no GPU. We trained the same tiny model (857,273 numbers) on both for a short time, 28 Oct.

16 GPUs

Tesla V100, 32 GB each. Your workspace: 4 CPU threads, no GPU.



The same training runs about 10 times faster on one GPU.

From sbatch to a GPU

You type one command in your workspace. The queue program on the server does the rest: it waits for a free GPU, runs your job there, and writes everything the job prints into a log file.

sbatch
you send the job

→

PENDING
it waits in the queue

→

RUNNING
1 GPU, 6 CPUs, 24 GB + your ~/work

→

DONE
or stopped at the time limit

The log: ~/work/jobs/<number>.log files your job saves land in your ~/work folder

NEW WORD
job = a program you send to run on the server

NEW WORD
queue = the waiting line for the GPUs

You do not wait at the screen: the queue runs your job when a GPU is free.

Three commands

The same names as on big research clusters. Type them in your workspace terminal, inside ~/work.

Command	What it does
sbatch job.sh	send a job script (a bash file)
sbatch --wrap "python3 train.py"	send one command, no script
squeue	show the line (* = your jobs)
scancel 97	stop job 97
tail -f ~/work/jobs/97.log	watch job 97 live (Ctrl+C stops watching)

sbatch sends, squeue shows, scancel stops.

Rules for sharing

One class, 16 GPUs. So that everyone gets a turn, the queue has three rules.

1 + 2

each person: 1 job running, 2 waiting

First turn

people with no running job go first.

15 min

time limit, then the job stops (--time 30 = max)

A 4th job says: **error: you already have 2 jobs waiting: wait or scancel one**

A queue makes sharing fair: nobody can take all the GPUs.

DEMO, 8 MINUTES

Live: three jobs, three sizes

On the screen: cd ~/work/slurm-demo, then bash demo.sh. Watch squeue, the logs, and the GPU board. Below: our test run, 1 minute per job on one V100 (lower loss = better).

Model	Numbers to learn	Letters / s	Best test loss
small	0.86 million	327,347	1.46
medium	4.9 million	279,925	1.42
large	25.5 million	66,660	**1.66, still going down**

> **A bigger model learns more slowly per second, but it can go further with more time.**

The real thing: Slurm

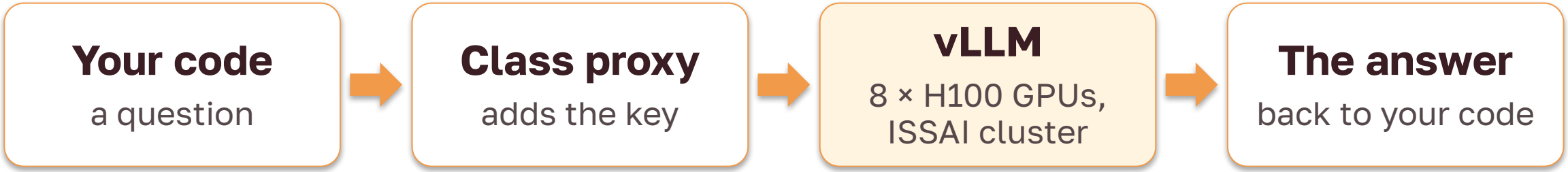
Most research clusters (thousands of GPUs) use a queue program called Slurm. Ours is a small copy with the same commands, so what you learn here works there.

	Our class queue	Slurm on a big cluster
Send	**sbatch job.sh**	**sbatch job.sh**
Time limit	**#SBATCH --time=6**	**#SBATCH --time=00:06:00**
Ask for a GPU	always 1 GPU	**#SBATCH --gres=gpu:1**
See the line	**squeue**	**squeue -u $USER**
The output	**~/work/jobs/97.log**	**slurm-97.out**

> **Same commands, bigger machine: this is how big models are trained.**

The model you call: served by vLLM

Training makes a model. Serving answers questions with it, for many people at once. This is the path of one call from your code to the class model (DeepSeek-V4.1-Flash).



2 servers. Each one is a **Slurm job** (8 hours), started with sbatch, like your jobs.

NEW WORD

serving = running a model so that it answers requests

Training makes the model. A server like vLLM lets many people use it at once.

Same model, same GPUs, 38 times faster

DeepSeek-V4.1-Flash on the same 8 H100 GPUs, one question at a time. Speed = tokens it writes per second (higher = faster). Our tests, 22 and 28 Oct.

Tokens per second, one question (higher = faster)



Reference code = the model's own simple program. **vLLM = a program built only to serve models fast.**

The way you run a model matters as much as the GPUs.

Many users at once: batching

Our test, 28 Oct: the same vLLM server, 1, 16 or 64 questions at the same time (200 tokens each). Higher = faster.

Questions at once	Total speed (all users)	Speed for each user
1	145 tokens/s	145 tokens/s
16	1,330 tokens/s	93 tokens/s
64	**3,501 tokens/s (24 ×)**	60 tokens/s

NEW WORD

batching = answering many requests together, in one pass

Together is much cheaper: 64 people at once, and each answer is still fast.

PART F: YOUR TURN

Try it yourself

On your own workspace, any time this week: about 15 minutes. These slides are your guide.

ON YOUR OWN: ABOUT 5 MINUTES

Train your own model

On your workspace (4 CPUs, no GPU), any time: train a tiny model on 134 poems of Abai. Write down its loss at the start and at the end.

1. Open your workspace. Click the terminal at the bottom.
2. `cd ~/work/day6`
3. `python3 train_tiny.py abai.txt`
4. Watch the loss and the text for 3 minutes.

Before you type: keyboard to English. Commands are in day6/README.md. ~ = Shift + key left of 1.

> **It stops by itself after 3 minutes. A problem? See “If something goes wrong” (2 slides on).**

REAL OUTPUT

What you will see

A real run in a class workspace, 16 Oct: in 2 min 40 s the loss went from 4.63 to 1.84, and the model began to write verse.

```
student@frp-stu-s92:~/work/day6$ python3 train_tiny.py abai.txt
text: 136,237 letters, alphabet: 82 letters, model: 830,802 numbers to learn, time: 3.0 min

0:00  step     0  loss 4.63  writes:  / иРж.:ІяпИТК!щңУІСтІһҚвкфу–квАүШЕюсДІқҰННңу / – ІсЯЛ;Ұтәәлющыгдқш»Е»нЕ»вНпвЯЫЛйО
Ш;ОрюР / вшМрұжЫсяц»бсНцХҮҚ–
0:20  step   173  loss 2.25  writes:  / Бір тені күр бастың олы шы ара дой ойың жүр. /  / Адабар тен дер жүнек босысы,
сып жала жоқ кетын / Кеті ке
0:40  step   345  loss 2.01  writes:  / Берінбайсың туым, / Жалығыз бесіз, ала бараса, жандай,  / Көзіні баршыртық жоқ
 / "йда қылы-сар.  / Айыны ш
1:00  step   513  loss 1.95  writes:  /  / Барға қылығы арға жаған, / Тамам бар дос ал жан бірекірмек. /  / Аял қартпад
аспа аламысын,  / Бәрі өстек
1:20  step   684  loss 1.88  writes:  / Ала да ол мақтау, салар бірдер қылмай бетіп. /  / Алысқанға, күлгін дүниет бард
ы жаған / Сап етіп жарқан та
1:40  step   858  loss 1.87  writes:  / Жемек кез бір мақтан. /  / Маңдан соң бабайласа, / әрі тоқтарасың, бұқпай / Тән
 ісінде алмай сөз. /  / Инді
2:00  step  1029  loss 1.85  writes:  / Таң-таң қалар жүр ма келер ел,  / Асық дағы қу темеге өзің ойлама. /  / Ант іс
күні кеш, қой таппа,  / әділ
2:20  step  1200  loss 1.89  writes:  / Міс құлықтың құдай,  / Айтқан сабырар. /  / Жауын жоқ,  / Сізді жоқ ол жоқ,  /
Жасырт жүрегімтің засы.  / Т
2:40  step  1374  loss 1.84  writes:  / Ағылған соң қайғы бар.  / Кейде орысам, жоқ,  / әзімсізді езімді текет,  / Мейі
нде мойын тапқан  / "Сөйлейд
```

Each line = 20 seconds:

step = rounds of learning so far

loss = mistake score, lower = better

writes = its text now

Every 20 seconds the loss goes down and the text gets better.

If something goes wrong

Read the last line of the error. Find it here. Do the fix, then run the command again.

You see	Do this
`No such file or directory`	You are in the wrong folder. Type **`cd ~/work/day6`**
`python: command not found`	Type **`python3`**, not python
~ or the letters look wrong	Switch the keyboard to English
`FileNotFoundError (abai_model.pt)`	Train first: **`python3 train_tiny.py abai.txt`**
It still runs	Wait 2-3 minutes, or press Ctrl+C to stop

> **Still stuck? Copy the error line to the class AI chat, or ask a mentor in class.**

ON YOUR OWN: 2 MINUTES EACH

Now practise with a score

Your model from step 1 (abai_model.pt) now practises with a score, like GRPO. Run line 1, then line 2. Write down what it writes each time.

1 `python3 rl_tiny.py abai.txt`

score = how many times “сөз” (word) is in its text. What happens?

2 `python3 rl_tiny.py abai.txt --stay 0.5`

--stay = how strongly it must stay close to the old model (0 = free)

Your own word: `--word жүрек` Our results: the next slide.

> **2 minutes each, on your own computer: watch what the score does.**

It learns what you score, not what you mean

Our results with rl_tiny.py on Abai's poems. Score = how many times “сөз” is in a text of the model.

Score only (20 seconds)

Сөз сөз сөз сөз сөз сөз сөз сөз сөз сөз ...

score 0 → 20: a top score, but no poem

Score + stay close (--stay 0.5)

Сенеуі жас сөздің келсең, /
Менен бөңке жасықпас едің бау.

score 0.6 → 1.8, still a poem

Real GRPO does the same: a score, plus “do not move too far”.

MORE TO TRY, IN ~/WORK/DAY6

Make it yours

Same code, your choices. Each line is one command. Compare the loss and the text with your first model.

`python3 train_tiny.py kk_wiki.txt`	Kazakh Wikipedia instead of poems
`python3 train_tiny.py shakespeare.txt`	English plays by Shakespeare
`python3 train_tiny.py abai.txt --minutes 5`	train longer: is the loss lower?
`python3 train_tiny.py abai.txt --write "Мен"`	your saved model continues your start

Your own text: put any .txt (50 KB or more) into ~/work/day6 and use its name.

> **Every model is different: it starts from different random knobs.**

SELF-CHECK: THINK FIRST

Train a model, or not?

4 project ideas. Think 1 minute: train a model for it, or not? The answers are on the next slide.

1

A bot that answers about our school timetable

2

A bot that talks like a pirate

3

A new task, no model can do it, 10,000 examples

4

The bot makes maths mistakes

First a better prompt, then better data. Train only when both fail.

Train or not: the answers

Cheap steps first: a better prompt (the bot's instruction text), then better data.

1 no

the timetable goes in the prompt

2 no

a good prompt is enough

3 yes

fine-tuning with your 10,000 examples

4 no

a reasoning model or a calculator tool

Most projects need a better prompt or better data, not a new model.

4 THINGS TO REMEMBER

1. A model learns by guessing and turning its knobs. The loss shows how well.
2. 3 stages: reading, school, practice with a score.
3. The data decides what the model becomes.
4. Different projects need different kinds of training, or none.

Sources

Our runs on the ISSAI FRP server (Tesla V100), 14 Oct 2026: tiny GPT on Kazakh Wikipedia (clean and dirty), Qwen3-0.6B-Base fine-tuning (SmolTalk), GRPO letter counting. Scripts: tools/.

Projects of this class, 26-28 Oct 2026: logs and files read by the mentors (no names).

Kazakh Wikipedia (CC BY-SA 4.0). Abai's poems: Wikisource (public domain). SmolTalk (Apache 2.0). Qwen3-0.6B (Apache 2.0).

Qwen3 Technical Report, arXiv 2505.09388 (2025). Common Crawl, commoncrawl.org. FineWeb / FineWeb-Edu, arXiv 2406.17557 (2024).

Kaplan et al. 2020 (arXiv 2001.08361) and Chinchilla 2022 (arXiv 2203.15556): compute ≈ 6 × parameters × tokens.

InstructGPT, Ouyang et al., arXiv 2203.02155 (2022). ISSAI KAZ-LLM, issai.nu.edu.kz/kazllm (Dec 2024). issai/Qwen3.5-9B-Base-Kazakh, Hugging Face (2026).

DeepSeek-R1, Nature 645, 633-638 (2025) and arXiv 2501.12948 v2. GRPO: DeepSeekMath, arXiv 2402.03300 (2024).